

GUÍA DE IMPLEMENTACIÓN · PASO A PASO

TaxVision — Construyendo el CRM Multitenant con .NET 10

Guía práctica de extremo a extremo: desde `dotnet new` hasta Kubernetes. Cada comando, cada archivo, cada decisión — explicando siempre el qué, el dónde y el porqué.

VS Code

.NET 10

DDD

CQRS

Wolverine

Mapster

YARP

RabbitMQ

SQL Server

EF Core

Docker

Kubernetes

Stack Vision Software

Manual de implementación de TaxVision · Santiago de los Caballeros, R.D.

Índice

0. Cómo usar esta guía	4
1. Prerrequisitos y entorno en VS Code	5
1.1 Instalar el SDK de .NET 10 y herramientas	5
1.2 Extensiones de VS Code	5
1.3 Verificar la instalación	6
2. Crear la solución y la estructura	7
2.1 La estructura de carpetas completa	7
2.2 Crear la solución y la jerarquía con comandos	8
2.3 Convenciones de nombres del proyecto	9
3. BuildingBlocks: el cimiento compartido	10
3.1 Crear el proyecto y sus paquetes	10
3.2 Result Pattern	11
3.3 BaseEntity, IDomainEvent, ITenantOwned	13
3.4 Repository y Unit of Work	14
3.5 Tenant Context y Correlation Context	15
3.6 Integration Events y middlewares compartidos	15
4. Tenant Service completo (la plantilla)	19
4.1 Crear los cuatro proyectos	19
4.2 La capa Domain	20
4.3 La capa Application (CQRS con Wolverine)	23
4.4 La capa Infrastructure (EF Core)	26
4.5 La capa Api (endpoints y arranque)	29
5. Auth Service (identidad y JWT)	33
5.2 Domain: el aggregate User	34
5.3 Infrastructure: hash de contraseñas y JWT	35
5.4 Application: registro y login	38
5.5 Refresh tokens	39
6. Customer Service (filtro multitenant)	40
6.1 Crear los proyectos y el dominio	40
6.2 El corazón del aislamiento: filtro global + estampado	41
6.3 El registro de infraestructura (con tenant)	44
7. Subscription, Notification y Billing	45
7.2 Subscription Service: planes y proration	45
7.3 Notification Service: consumidor puro	46
7.4 Billing Service: idempotencia con dinero	47
8. API Gateway con YARP	49

8.2 Configurar rutas y clusters	49
8.3 La propagación del tenant (seguridad)	51
8.4 El Program.cs del gateway	52
9. Wolverine + RabbitMQ (eventos)	54
9.2 Configurar Wolverine con RabbitMQ y Outbox	54
9.3 Consumir el evento en otro servicio	55
9.4 Por qué Outbox e Inbox importan tanto	56
10. Migraciones de base de datos	57
10.1 Crear la primera migración	57
10.3 Migraciones en producción	58
11. Docker: empaquetar y levantar todo	60
11.1 El Dockerfile de un microservicio	60
11.2 El docker-compose para desarrollo	61
11.3 Levantar y verificar	63
12. Kubernetes: desplegar en producción	65
12.1 Namespace, ConfigMap y Secret	65
12.2 Deployment y Service de un microservicio	65
12.3 El gateway y el Ingress	67
12.4 Escalado automático (HPA)	67
12.5 Migraciones como Job	68
Cierre: lo que has construido	70

SECCIÓN 0

Cómo usar esta guía

Esta guía es el complemento práctico del libro de arquitectura. Si el libro responde "¿por qué se diseña así?", esta guía responde "¿cómo lo construyo yo, ahora, en mi máquina, con VS Code y .NET 10?".

Construiremos **TaxVision**, un CRM multitenant para oficinas, como una solución de microservicios. Iremos de cero a un sistema desplegable, sin saltarnos pasos. Cada sección sigue este patrón:

- `ruta/del/archivo.cs` — te dice **dónde** va exactamente cada archivo en la estructura.
- Bloques de **terminal** con los comandos exactos a ejecutar (con el prefijo `$`).
- El **código completo** de cada archivo, comentado.
- Una caja `¿Por qué?` que explica la decisión de diseño detrás de cada paso.

Una convención importante. Todos los comandos se ejecutan desde la **terminal integrada de VS Code** (`Ctrl/Cmd + N` o *Terminal* → *New Terminal*), situándote en la carpeta raíz del proyecto salvo que se indique lo contrario. Los identificadores del código van en inglés (estándar del equipo); las explicaciones, en español.

SECCIÓN 1

Prerrequisitos y entorno en VS Code

Antes de escribir código, dejamos el entorno listo. Trabajaremos íntegramente con la CLI de .NET (`dotnet`) y VS Code, sin Visual Studio. Esto nos da un flujo reproducible, ideal para documentar y para que cualquier miembro del equipo lo replique igual.

1.1 Instalar el SDK de .NET 10 y herramientas

► Paso 1 — Instalar el SDK de .NET 10

Descarga e instala el SDK desde el sitio oficial de .NET. Tras instalarlo, verifica la versión:

```
$ dotnet --version
10.0.100

# Ver todos los SDKs instalados
$ dotnet --list-sdks
10.0.100 [C:\Program Files\dotnet\sdk]
```

► Paso 2 — Instalar la herramienta de Entity Framework Core

La usaremos para crear y aplicar migraciones de base de datos. Se instala como herramienta global:

```
$ dotnet tool install --global dotnet-ef
.

# Verificar
$ dotnet ef --version
Entity Framework Core .NET Command-line Tools 10.0.0
```

¿Por qué? — ¿Por qué la CLI y no Visual Studio? La CLI hace cada paso explícito y scriptable. Lo que documentamos aquí es exactamente lo que se ejecuta —sin menús ni asistentes ocultos—, por lo que la guía es 100% reproducible y sirve igual en Windows, macOS o Linux. Además, es el flujo que usará el pipeline de CI/CD en el servidor.

1.2 Extensiones de VS Code

Instala estas extensiones desde el panel de extensiones (**Ctrl/Cmd + Shift + X**):

Extensión	Para qué sirve
C# Dev Kit (Microsoft)	IntelliSense, depuración, explorador de soluciones y tests para C#.
C# (Microsoft)	El motor de lenguaje base (se instala con el Dev Kit).
Docker (Microsoft)	Gestionar imágenes y contenedores, sintaxis de Dockerfile y compose.
Kubernetes (Microsoft)	Gestionar manifiestos YAML y clústeres.
YAML (Red Hat)	Validación y autocompletado de los manifiestos de K8s y compose.
REST Client o Thunder Client	Probar los endpoints HTTP sin salir del editor.

1.3 Verificar la instalación

Comprueba que tienes Docker funcionando (lo necesitaremos para SQL Server, RabbitMQ y el despliegue):

```
$ docker --version
Docker version 27.x
$ docker compose version
Docker Compose version v2.x
```

Listo para empezar. Si los cuatro comandos (`dotnet` , `dotnet ef` , `docker` , `docker compose`) responden con su versión, el entorno está completo. No necesitas instalar SQL Server ni RabbitMQ localmente: los levantaremos en contenedores en la sección 11.

SECCIÓN 2

Crear la solución y la estructura

Vamos a crear el esqueleto completo de TaxVision: la solución, la estructura de carpetas y los proyectos base. Primero verás el mapa de carpetas completo para tener la foto mental, y luego los comandos exactos para construirlo.

2.1 La estructura de carpetas completa

Esta es la organización que tendrá TaxVision. Una solución, varias carpetas de microservicio (cada una con sus 4 proyectos de Clean Architecture), una librería compartida, el gateway, y carpetas de infraestructura (Docker, K8s).

```
TaxVision/
├── TaxVision.sln
├── src/
│   ├── BuildingBlocks/
│   │   └── BuildingBlocks.csproj
│   ├── Services/
│   │   ├── Tenant/
│   │   │   ├── TaxVision.Tenant.Domain/
│   │   │   ├── TaxVision.Tenant.Application/
│   │   │   ├── TaxVision.Tenant.Infrastructure/
│   │   │   └── TaxVision.Tenant.Api/
│   │   ├── Auth/
│   │   ├── Customer/
│   │   ├── Subscription/
│   │   ├── Notification/
│   │   └── Billing/
│   └── Gateway/
│       └── TaxVision.Gateway/
├── tests/
│   ├── TaxVision.Tenant.UnitTests/
│   └── TaxVision.Tenant.IntegrationTests/
├── deploy/
│   ├── docker/
│   │   └── docker-compose.yml
│   └── k8s/
├── .gitignore
├── .env
└── global.json
```

la solución que agrupa todo

código transversal compartido

microservicio de Tenants

Auth Service (misma estructura x4)

Customer Service

Subscription Service

Notification Service

Billing Service

API Gateway con YARP

proyectos de pruebas

entorno de desarrollo completo

manifiestos de Kubernetes

variables locales (NO se versiona)

fija la versión del SDK

¿Por qué? — ¿Por qué esta estructura? Separar `src/`, `tests/` y `deploy/` mantiene limpio el repositorio. Dentro de `src/Services/`, cada microservicio es una carpeta autocontenida con sus 4 proyectos (Domain, Application, Infrastructure, Api): así un servicio es una "rebanada vertical" completa del negocio, y puedes razonar sobre él sin mirar a los demás. El gateway y BuildingBlocks viven aparte porque tienen roles transversales.

2.2 Crear la solución y la jerarquía con comandos

► Paso 1 — Crear la carpeta raíz y la solución

```
# Crear y entrar en la carpeta raíz
$ mkdir TaxVision && cd TaxVision

# Crear la solución (.sln)
$ dotnet new sln --name TaxVision

# Abrir VS Code en esta carpeta
$ code .
```

► Paso 2 — Fijar la versión del SDK con global.json

Esto garantiza que todo el equipo compile con la misma versión del SDK.

```
$ dotnet new globaljson --sdk-version 10.0.100
```

► Paso 3 — Crear las carpetas de la estructura

```
# Carpetas principales
$ mkdir -p src/BuildingBlocks
$ mkdir -p src/Services/Tenant src/Services/Auth src/Services/Customer
$ mkdir -p src/Services/Subscription src/Services/Notification src/Services/Billing
$ mkdir -p src/Gateway
$ mkdir -p deploy/docker deploy/k8s tests
```

► Paso 4 — Crear el .gitignore

```
# .gitignore estándar para proyectos .NET
$ dotnet new gitignore
```

Y añada manualmente estas líneas al final del `.gitignore` para no versionar secretos ni artefactos locales:

TaxVision/.gitignore (añadir al final)

```
# Secretos y entorno local
.env
*.user
appsettings.*.local.json
```

No versiones el archivo .env. Contendrá contraseñas y claves. La regla de oro: ningún secreto en el repositorio, nunca. En desarrollo van en `.env` (ignorado por git); en producción, en secretos de Kubernetes (sección 12).

2.3 Convenciones de nombres del proyecto

Mantendremos estas convenciones de forma estricta en todo TaxVision:

Elemento	Convención	Ejemplo
Proyecto	<code>TaxVision.{Servicio}.{Capa}</code>	<code>TaxVision.Tenant.Domain</code>
Command	Verbo imperativo + <code>Command</code>	<code>CreateTenantCommand</code>
Query	Verbo + <code>Query</code>	<code>GetTenantsQuery</code>
Evento de dominio	Pasado + <code>DomainEvent</code>	<code>TenantCreatedDomainEvent</code>
Evento de integración	Pasado + <code>IntegrationEvent</code>	<code>TenantCreatedIntegrationEvent</code>
Handler	Nombre del mensaje + <code>Handler</code>	<code>CreateTenantHandler</code>
Carpetas internas	Por <i>feature</i> , no por tipo	<code>Tenants/</code> , no <code>Controllers/</code>

Resumen de la sección 2

- Creamos la solución `TaxVision.sln` y la estructura `src/ tests/ deploy/`.
- Cada microservicio vivirá en `src/Services/{Nombre}/` con sus **4 proyectos** de Clean Architecture.
- `global.json` fija el SDK; el `.gitignore` protege secretos.
- Seguimos **convenciones de nombres estrictas** en toda la solución.

SECCIÓN 3

BuildingBlocks: el cimiento compartido

Antes de construir ningún microservicio, creamos `BuildingBlocks`: la librería con el código que todos comparten. Empezamos por aquí porque cada servicio dependerá de ella. Contiene abstracciones genéricas (Result, BaseEntity, interfaces de persistencia, tenant context, eventos), nunca lógica de un dominio concreto.

3.1 Crear el proyecto y sus paquetes

► Paso 1 — Crear el proyecto de librería de clases

```
# Desde la raíz TaxVision/  
$ dotnet new classlib -n BuildingBlocks -o src/BuildingBlocks  
  
# Agregarlo a la solución  
$ dotnet sln add src/BuildingBlocks/BuildingBlocks.csproj
```

► Paso 2 — Borrar la clase de ejemplo y crear las carpetas

```
$ rm src/BuildingBlocks/Class1.cs  
$ cd src/BuildingBlocks  
$ mkdir Domain Results Persistence Tenancy Messaging Middleware Common  
$ cd ../../
```

► Paso 3 — Instalar los paquetes NuGet necesarios

```
# Abstracciones que necesitan los middlewares y el logging  
$ cd src/BuildingBlocks  
$ dotnet add package Microsoft.AspNetCore.Http.Abstractions  
$ dotnet add package Microsoft.Extensions.Logging.Abstractions  
$ dotnet add package Serilog.AspNetCore  
$ cd ../../
```

El archivo de proyecto debe apuntar a .NET 10. Verifica que `BuildingBlocks.csproj` tenga el target correcto:

```
src/BuildingBlocks/BuildingBlocks.csproj
```

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

¿Por qué? — ¿Por qué Nullable e ImplicitUsings activados? Nullable hace que el compilador te avise de posibles referencias nulas en compilación, eliminando una clase entera de bugs (`NullReferenceException`). ImplicitUsings incluye automáticamente los `using` más comunes, reduciendo ruido en cada archivo. Ambos son estándar en proyectos .NET modernos.

3.2 Result Pattern

El primer bloque: devolver resultados explícitos en lugar de lanzar excepciones para errores esperables (validación, no encontrado). Hace visible el flujo de error en la firma de cada método.

`src/BuildingBlocks/Results/Error.cs`

```
namespace BuildingBlocks.Results;

// Representa un error de negocio con un código estable y un mensaje legible.
// El Code (ej. "Tenant.SubdomainTaken") es ideal para traducir o testear.
public sealed record Error(string Code, string Message)
{
    public static readonly Error None = new(string.Empty, string.Empty);
}
```

`src/BuildingBlocks/Results/Result.cs`

```

namespace BuildingBlocks.Results;

// Resultado de una operación SIN valor de retorno (solo éxito/fracaso).
public class Result
{
    protected Result(bool isSuccess, Error error)
    {
        // Invariante: éxito sin error, o fracaso con error. Nunca mezclados.
        if (isSuccess && error != Error.None)
            throw new InvalidOperationException("Un éxito no puede llevar error.");
        if (!isSuccess && error == Error.None)
            throw new InvalidOperationException("Un fracaso requiere un error.");

        IsSuccess = isSuccess;
        Error = error;
    }

    public bool IsSuccess { get; }
    public bool IsFailure => !IsSuccess;
    public Error Error { get; }

    public static Result Success() => new(true, Error.None);
    public static Result Failure(Error error) => new(false, error);

    // Versiones genéricas para resultados CON valor.
    public static Result<T> Success<T>(T value) => new(value, true, Error.None);
    public static Result<T> Failure<T>(Error error) => new(default!, false, error);
}

// Resultado de una operación que devuelve un valor en caso de éxito.
public sealed class Result<T> : Result
{
    private readonly T _value;

    internal Result(T value, bool isSuccess, Error error)
        : base(isSuccess, error) => _value = value;

    // Acceder al valor de un resultado fallido es un error de programación.
    public T Value => IsSuccess
        ? _value
        : throw new InvalidOperationException("No hay valor en un Result fallido.");
}

```

¿Por qué? — ¿Por qué Result en vez de excepciones? Las excepciones son caras y, sobre todo, *invisibles*: nada en la firma de un método te dice que puede fallar. Con `Result<T>`, el fallo es parte del tipo de retorno: el compilador y el lector ven que hay que manejar el caso de error. Reservamos las excepciones para fallos *inesperados* (un bug, la base de datos caída), no para reglas de negocio normales como "el email ya existe".

3.3 BaseEntity, IDomainEvent, ITenantOwned

La base de todas las entidades del dominio: identidad, acumulación de eventos de dominio, y el contrato para entidades que pertenecen a un tenant.

src/BuildingBlocks/Domain/IDomainEvent.cs

```
namespace BuildingBlocks.Domain;

// Marca un evento de dominio: un hecho relevante ocurrido DENTRO del servicio.
public interface IDomainEvent
{
    DateTime OccurredOn { get; }
}
```

src/BuildingBlocks/Domain/BaseEntity.cs

```
namespace BuildingBlocks.Domain;

// Clase base de toda entidad: gestiona identidad y eventos de dominio.
public abstract class BaseEntity
{
    private readonly List<IDomainEvent> _domainEvents = new();

    // Identidad: un Guid generado al crear la entidad.
    public Guid Id { get; protected set; } = Guid.NewGuid();

    // Solo lectura desde fuera: los eventos se añaden con Raise().
    public IReadOnlyCollection<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();

    // Los hijos levantan eventos cuando ocurre algo relevante.
    protected void Raise(IDomainEvent domainEvent) => _domainEvents.Add(domainEvent);

    // Se limpian tras publicarlos (lo hace la capa de Infrastructure).
    public void ClearDomainEvents() => _domainEvents.Clear();
}
```

src/BuildingBlocks/Domain/ITenantOwned.cs

```
namespace BuildingBlocks.Domain;

// Contrato de toda entidad que pertenece a un tenant. Permite al DbContext
// estampar el TenantId automáticamente al guardar (ver Infrastructure).
public interface ITenantOwned
{
    Guid TenantId { get; }
    void SetTenant(Guid tenantId);
}
```

¿Por qué? — ¿Por qué los eventos viven en la entidad? Cuando algo importante ocurre en el dominio (un tenant se crea), la propia entidad "sabe" que debe anunciarlo. Acumularlos en `BaseEntity` y publicarlos *después* de persistir (no durante) garantiza que solo se anuncian hechos que de verdad se guardaron. Es la semilla del patrón Outbox que Wolverine gestionará en la sección 9.

3.4 Repository y Unit of Work

Interfaces de persistencia. Aquí solo el contrato; cada servicio lo implementará con EF Core en su capa de Infrastructure.

`src/BuildingBlocks/Persistence/IRepository.cs`

```
using BuildingBlocks.Domain;

namespace BuildingBlocks.Persistence;

// Abstracción de acceso a aggregates, como si fueran una colección en memoria.
public interface IRepository<T> where T : BaseEntity
{
    Task<T?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task AddAsync(T entity, CancellationToken ct = default);
    void Remove(T entity);
}
```

`src/BuildingBlocks/Persistence/IUnitOfWork.cs`

```
namespace BuildingBlocks.Persistence;

// Confirma todos los cambios pendientes en UNA transacción.
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync(CancellationToken ct = default);
}
```

¿Por qué? — ¿Por qué la interfaz aquí y la implementación en cada servicio? El contrato (qué se puede hacer) es genérico y compartido. La implementación (cómo se hace con EF Core) depende del `DbContext` concreto de cada servicio. Separarlos permite que el dominio y la aplicación dependan solo de la abstracción —lo que los hace testeables con dobles (sección de tests)— y que la tecnología de persistencia sea un detalle intercambiable.

3.5 Tenant Context y Correlation Context

Dos servicios *scoped* (una instancia por petición) que transportan, respectivamente, el tenant actual y el id de correlación a lo largo de toda la petición.

src/BuildingBlocks/Tenancy/ITenantContext.cs

```
namespace BuildingBlocks.Tenancy;

// Acceso tipado y seguro al tenant de la petición actual.
public interface ITenantContext
{
    Guid TenantId { get; }
    bool HasTenant { get; }
}

public sealed class TenantContext : ITenantContext
{
    private Guid? _tenantId;

    public Guid TenantId =>
        _tenantId ?? throw new InvalidOperationException(
            "No hay tenant en el contexto actual.");

    public bool HasTenant => _tenantId.HasValue;

    // Lo llama el middleware de resolución de tenant al inicio de la petición.
    public void SetTenant(Guid tenantId) => _tenantId = tenantId;
}
```

src/BuildingBlocks/Common/ICorrelationContext.cs

```
namespace BuildingBlocks.Common;

// Transporta el id de correlación para rastrear una operación entre servicios.
public interface ICorrelationContext
{
    string CorrelationId { get; }
    void Set(string correlationId);
}

public sealed class CorrelationContext : ICorrelationContext
{
    public string CorrelationId { get; private set; } = string.Empty;
    public void Set(string id) => CorrelationId = id;
}
```

3.6 Integration Events y middlewares compartidos

El contrato base de los eventos de integración (que cruzan servicios) y los middlewares transversales que cada servicio enchufará en su pipeline.

src/BuildingBlocks/Messaging/IntegrationEvent.cs

```

namespace BuildingBlocks.Messaging;

// Contrato: todo evento de integración lleva trazabilidad y tenant.
public interface IIntegrationEvent
{
    Guid EventId { get; }
    Guid TenantId { get; }
    DateTime OccurredOn { get; }
    string CorrelationId { get; }
}

// Base con valores por defecto. Los eventos concretos heredan de aquí.
public abstract record IntegrationEvent : IIntegrationEvent
{
    public Guid EventId { get; init; } = Guid.NewGuid();
    public Guid TenantId { get; init; }
    public DateTime OccurredOn { get; init; } = DateTime.UtcNow;
    public string CorrelationId { get; init; } = string.Empty;
}

```

src/BuildingBlocks/Middleware/CorrelationIdMiddleware.cs

```

using BuildingBlocks.Common;
using Microsoft.AspNetCore.Http;
using Serilog.Context;

namespace BuildingBlocks.Middleware;

// Asigna/propaga el correlation id y lo inyecta en el scope de logging.
public sealed class CorrelationIdMiddleware(RequestDelegate next)
{
    public const string Header = "X-Correlation-Id";

    public async Task InvokeAsync(HttpContext ctx, ICorrelationContext corr)
    {
        // Reusa el id del gateway si llega; si no, genera uno nuevo.
        var id = ctx.Request.Headers[Header].FirstOrDefault()
            ?? Guid.NewGuid().ToString();
        ((CorrelationContext)corr).Set(id);
        ctx.Response.Headers[Header] = id;

        // Todo log dentro de esta petición incluirá el CorrelationId.
        using (LogContext.PushProperty("CorrelationId", id))
            await next(ctx);
    }
}

```

src/BuildingBlocks/Middleware/TenantResolutionMiddleware.cs


```

using BuildingBlocks.Tenancy;
using Microsoft.AspNetCore.Http;

namespace BuildingBlocks.Middleware;

// Lee X-Tenant-Id (que el gateway garantiza desde el token) y la vuelca
// en el TenantContext. A partir de aquí, EF Core filtra por ese tenant.
public sealed class TenantResolutionMiddleware(RequestDelegate next)
{
    public async Task InvokeAsync(HttpContext ctx, TenantContext tenant)
    {
        var raw = ctx.Request.Headers["X-Tenant-Id"].FirstOrDefault();
        if (Guid.TryParse(raw, out var tenantId))
            tenant.SetTenant(tenantId);

        await next(ctx);
    }
}

```

src/BuildingBlocks/Middleware/ExceptionHandlerMiddleware.cs

```

using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;

namespace BuildingBlocks.Middleware;

// Captura excepciones no controladas: registra el detalle y devuelve
// un error genérico y seguro al cliente (sin filtrar el stack trace).
public sealed class ExceptionHandlingMiddleware(
    RequestDelegate next,
    ILogger<ExceptionHandlerMiddleware> logger)
{
    public async Task InvokeAsync(HttpContext ctx)
    {
        try
        {
            await next(ctx);
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Error no controlado en {Path}", ctx.Request.Path);
            ctx.Response.StatusCode = StatusCodes.Status500InternalServerError;
            await ctx.Response.WriteAsJsonAsync(new ProblemDetails
            {
                Status = 500,
                Title = "Error interno",
                Detail = "Ocurrió un error. Use el id de correlación para soporte."
            });
        }
    }
}

```

Finalmente, una clase de extensión que registra todo lo de BuildingBlocks de una sola línea en cada servicio:

src/BuildingBlocks/Common/DependencyInjection.cs

```

using BuildingBlocks.Tenancy;
using Microsoft.Extensions.DependencyInjection;

namespace BuildingBlocks.Common;

public static class BuildingBlocksRegistration
{
    // Registra los servicios scoped compartidos. Se llama desde cada Program.cs.
    public static IServiceCollection AddBuildingBlocks(this IServiceCollection services)
    {
        // Scoped = una instancia por petición HTTP.
        services.AddScoped<TenantContext>();
        services.AddScoped<ITenantContext>(sp => sp.GetRequiredService<TenantContext>());
        services.AddScoped<CorrelationContext>();
        services.AddScoped<ICorrelationContext>(sp => sp.GetRequiredService<CorrelationContext>(
    ));
        return services;
    }
}

```

¿Por qué? — ¿Por qué registrar `TenantContext` como `scoped` y exponerlo por dos vías? Scoped garantiza que el tenant resuelto al inicio de una petición sea el mismo durante toda esa petición (y distinto para peticiones concurrentes de otros tenants). Lo registramos como clase concreta (para que el middleware llame a `SetTenant`) y como interfaz (para que el resto del código dependa solo de `ITenantContext`, sin poder modificarlo). Misma instancia, dos puertas: una de escritura, otra de solo lectura.

► Compila para verificar

```

$ dotnet build src/BuildingBlocks
Build succeeded.

```

Resumen de la sección 3

- Creamos `BuildingBlocks` con `dotnet new classlib` y lo añadimos a la solución.
- Implementamos el **Result Pattern**, `BaseEntity` / `IDomainEvent` / `ITenantOwned`, las interfaces de **repositorio** y **unit of work**, el **tenant** y **correlation context**, el contrato de **integration events** y los **middlewares compartidos**.
- Una extensión `AddBuildingBlocks()` registra todo en cada servicio con una línea.
- Cada pieza es una **abstracción genérica**: ninguna conoce un dominio concreto.

SECCIÓN 4

Tenant Service completo (la plantilla)

Esta es la sección más importante de la guía. Construiremos el Tenant Service de principio a fin: los cuatro proyectos, el dominio, la aplicación con CQRS sobre Wolverine, la infraestructura con EF Core, y la API con sus endpoints. Una vez lo entiendas, los demás servicios son variaciones del mismo patrón.

El Tenant Service gobierna el ciclo de vida de los tenants (las oficinas que usan TaxVision). Su evento `TenantCreated` dispara el aprovisionamiento del resto del sistema, así que es el lugar natural para empezar.

4.1 Crear los cuatro proyectos

► Paso 1 — Crear los proyectos de cada capa

Cada microservicio son cuatro proyectos. Domain, Application e Infrastructure son librerías de clases; Api es una Web API.

```
# Desde la raíz TaxVision/
$ dotnet new classlib -n TaxVision.Tenant.Domain -o src/Services/Tenant/
TaxVision.Tenant.Domain
$ dotnet new classlib -n TaxVision.Tenant.Application -o src/Services/Tenant/
TaxVision.Tenant.Application
$ dotnet new classlib -n TaxVision.Tenant.Infrastructure -o src/Services/Tenant/
TaxVision.Tenant.Infrastructure
$ dotnet new web -n TaxVision.Tenant.Api -o src/Services/Tenant/
TaxVision.Tenant.Api

# Borrar las clases de ejemplo
$ rm src/Services/Tenant/TaxVision.Tenant.Domain/Class1.cs
$ rm src/Services/Tenant/TaxVision.Tenant.Application/Class1.cs
$ rm src/Services/Tenant/TaxVision.Tenant.Infrastructure/Class1.cs
```

► Paso 2 — Agregarlos a la solución

```
$ dotnet sln add src/Services/Tenant/TaxVision.Tenant.Domain/TaxVision.Tenant.Domain.csproj
$ dotnet sln add src/Services/Tenant/TaxVision.Tenant.Application/
TaxVision.Tenant.Application.csproj
$ dotnet sln add src/Services/Tenant/TaxVision.Tenant.Infrastructure/
TaxVision.Tenant.Infrastructure.csproj
$ dotnet sln add src/Services/Tenant/TaxVision.Tenant.Api/TaxVision.Tenant.Api.csproj
```

► Paso 3 — Configurar las referencias entre proyectos

Aquí se materializa la regla de oro de Clean Architecture: **las dependencias apuntan hacia el dominio**. Domain no referencia a nadie (salvo BuildingBlocks); Application referencia a Domain; Infrastructure a Application; Api a todas.

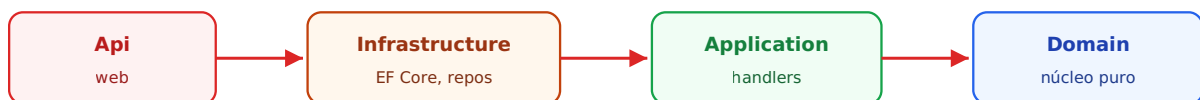
```
# Domain → BuildingBlocks
$ dotnet add src/Services/Tenant/TaxVision.Tenant.Domain reference src/BuildingBlocks/BuildingBlocks.csproj

# Application → Domain
$ dotnet add src/Services/Tenant/TaxVision.Tenant.Application reference src/Services/Tenant/TaxVision.Tenant.Domain

# Infrastructure → Application (y transitivamente Domain)
$ dotnet add src/Services/Tenant/TaxVision.Tenant.Infrastructure reference src/Services/Tenant/TaxVision.Tenant.Application

# Api → Application + Infrastructure
$ dotnet add src/Services/Tenant/TaxVision.Tenant.Api reference src/Services/Tenant/TaxVision.Tenant.Application
$ dotnet add src/Services/Tenant/TaxVision.Tenant.Api reference src/Services/Tenant/TaxVision.Tenant.Infrastructure
```

¿Por qué? — ¿Por qué Domain no referencia a Infrastructure? Si el dominio dependiera de la base de datos, sería imposible probarlo sin ella y quedaría atado a una tecnología concreta. Al invertir la dependencia (Infrastructure conoce a Domain, no al revés), el dominio queda puro: solo reglas de negocio, testeable en milisegundos, e independiente de si detrás hay SQL Server, otra base o un mock. Esto es el corazón de la Arquitectura Limpia.



"referencia a" → · Las flechas apuntan hacia el dominio. Nunca al revés.

Figura 4.1 — Las referencias entre proyectos del Tenant Service apuntan siempre hacia el dominio.

4.2 La capa Domain

El corazón del servicio. Aquí viven las reglas de negocio del tenant, sin una sola línea de infraestructura.

► Crear las carpetas del dominio

```
$ cd src/Services/Tenant/TaxVision.Tenant.Domain  
$ mkdir -p Tenants/Events  
$ cd ../../../../..
```

El aggregate Tenant

```
src/Services/Tenant/TaxVision.Tenant.Domain/Tenants/Tenant.cs
```

```

using System.Text.RegularExpressions;
using BuildingBlocks.Domain;
using BuildingBlocks.Results;
using TaxVision.Tenant.Domain.Tenants.Events;

namespace TaxVision.Tenant.Domain.Tenants;

// Estados posibles de un tenant a lo largo de su vida.
public enum TenantStatus { Active, Suspended, Closed }

// Aggregate Root: la única puerta de entrada a este agregado.
// Constructor privado + fábrica estática = imposible crear un tenant inválido.
public sealed class Tenant : BaseEntity
{
    // Propiedades con setter privado: solo el propio aggregate se modifica a sí mismo.
    public string Name { get; private set; } = default!;
    public string Subdomain { get; private set; } = default!;
    public TenantStatus Status { get; private set; }
    public DateTime CreatedAtUtc { get; private set; }

    // EF Core necesita un constructor sin parámetros (lo usa al materializar).
    private Tenant() { }

    // FÁBRICA: la única forma de crear un Tenant. Valida y devuelve Result.
    public static Result<Tenant> Create(string name, string subdomain)
    {
        if (string.IsNullOrEmpty(name))
            return Result.Failure<Tenant>(
                new Error("Tenant.Name", "El nombre es obligatorio."));

        var sub = (subdomain ?? "").Trim().ToLowerInvariant();

        // Regla de negocio: el subdominio será parte de una URL, así que se
        // restringe a minúsculas, dígitos y guiones, de 3 a 40 caracteres.
        if (!Regex.IsMatch(sub, "^[a-z0-9-]{3,40}$"))
            return Result.Failure<Tenant>(
                new Error("Tenant.Subdomain", "Subdominio inválido (a-z, 0-9, guiones, 3-40)."));
    }

    var tenant = new Tenant
    {
        Name = name.Trim(),
        Subdomain = sub,
        Status = TenantStatus.Active,
        CreatedAtUtc = DateTime.UtcNow
    };

    // Levanta un evento de dominio: "este tenant fue creado".
    tenant.Raise(new TenantCreatedDomainEvent(tenant.Id, tenant.Subdomain));
    return Result.Success(tenant);
}

// Comportamiento con intención de negocio, NO un setter público.
public Result Suspend()
{
    if (Status != TenantStatus.Active)
        return Result.Failure(
            new Error("Tenant.NotActive", "Solo se suspende un tenant activo."));

    Status = TenantStatus.Suspended;
}

```

```

        Raise(new TenantSuspendedDomainEvent(Id));
        return Result.Success();
    }
}

```

Los eventos de dominio

src/Services/Tenant/TaxVision.Tenant.Domain/Tenants/Events/TenantCreatedDomainEvent.cs

```

using BuildingBlocks.Domain;

namespace TaxVision.Tenant.Domain.Tenants.Events;

// Hecho pasado, immutable. Se nombra en pasado: "Created".
public sealed record TenantCreatedDomainEvent(Guid TenantId, string Subdomain)
    : IDomainEvent
{
    public DateTime OccurredOn { get; } = DateTime.UtcNow;
}

public sealed record TenantSuspendedDomainEvent(Guid TenantId)
    : IDomainEvent
{
    public DateTime OccurredOn { get; } = DateTime.UtcNow;
}

```

¿Por qué? — ¿Por qué setters privados y métodos como `Suspend()`? Si `Status` tuviera un setter público, cualquiera podría escribir `tenant.Status = Suspended` saltándose la regla "solo se suspende un activo". Al exponer comportamiento (`Suspend()`) en vez de datos, el aggregate *protege sus propias invariantes*: nadie puede dejarlo en un estado inválido. Esto evita el "modelo anémico" (entidades que son solo bolsas de datos con la lógica fuera).

4.3 La capa Application (CQRS con Wolverine)

Aquí viven los casos de uso: comandos, queries y sus handlers. Wolverine los descubre por convención y los despacha.

► Paso 1 — Instalar paquetes y crear carpetas

```

$ cd src/Services/Tenant/TaxVision.Tenant.Application
$ dotnet add package WolverineFx
$ dotnet add package Mapster
$ mkdir -p Tenants/Commands Tenants/Queries Tenants/IntegrationEvents Abstractions
$ cd ../../../../..

```

La abstracción del repositorio específico

El repositorio genérico de BuildingBlocks no tiene un método para comprobar subdominios. Lo extendemos con una interfaz específica del servicio.

```
src/Services/Tenant/TaxVision.Tenant.Application/Abstractions/ITenantRepository.cs
```

```
using BuildingBlocks.Persistence;
using TaxVision.Tenant.Domain.Tenants;

namespace TaxVision.Tenant.Application.Abstractions;

// Hereda las operaciones genéricas y añade una consulta propia del dominio.
public interface ITenantRepository : IRepository<Domain.Tenants.Tenant>
{
    Task<bool> SubdomainExistsAsync(string subdomain, CancellationToken ct = default);
}
```

El evento de integración

```
src/Services/Tenant/TaxVision.Tenant.Application/Tenants/IntegrationEvents/
TenantCreatedIntegrationEvent.cs
```

```
using BuildingBlocks.Messaging;

namespace TaxVision.Tenant.Application.Tenants.IntegrationEvents;

// El evento que CRUZA a otros servicios. Es un contrato público y estable.
// Lleva lo mínimo que Auth, Subscription, Notification... necesitan para arrancar.
public sealed record TenantCreatedIntegrationEvent : IntegrationEvent
{
    public required Guid NewTenantId { get; init; }
    public required string Name { get; init; }
    public required string Subdomain { get; init; }
    public required string AdminEmail { get; init; }
}
```

El comando y su handler

```
src/Services/Tenant/TaxVision.Tenant.Application/Tenants/Commands/CreateTenant.cs
```



```

using BuildingBlocks.Results;
using TaxVision.Tenant.Application.Abstractions;
using TaxVision.Tenant.Application.Tenants.IntegrationEvents;
using Wolverine;
using DomainTenant = TaxVision.Tenant.Domain.Tenants.Tenant;

namespace TaxVision.Tenant.Application.Tenants.Commands;

// COMMAND: intención de cambiar el estado. Datos mínimos para crear el tenant.
public sealed record CreateTenantCommand(
    string Name, string Subdomain, string AdminEmail);

// La respuesta que devolvemos al cliente.
public sealed record TenantResponse(Guid Id, string Name, string Subdomain);

// HANDLER: Wolverine lo descubre por convención (método estático Handle).
// Las dependencias se inyectan como parámetros del método.
public static class CreateTenantHandler
{
    public static async Task<Result<TenantResponse>> Handle(
        CreateTenantCommand cmd,
        ITenantRepository repo,
        IMessageBus bus, // bus de Wolverine para publicar
        CancellationToken ct)
    {
        // 1) Regla que cruza la persistencia: subdominio único.
        if (await repo.SubdomainExistsAsync(cmd.Subdomain, ct))
            return Result.Failure<TenantResponse>(
                new Error("Tenant.SubdomainTaken", "El subdominio ya está en uso."));

        // 2) Crear el aggregate vía su fábrica (valida formato e invariantes).
        var result = DomainTenant.Create(cmd.Name, cmd.Subdomain);
        if (result.IsFailure)
            return Result.Failure<TenantResponse>(result.Error);

        var tenant = result.Value;

        // 3) Persistir. (El AddAsync registra; el commit lo hace Wolverine
        // con su transacción + Outbox: ver sección 9.)
        await repo.AddAsync(tenant, ct);

        // 4) Publicar el evento de integración → dispara la coreografía.
        // Wolverine lo mete en el Outbox dentro de la misma transacción.
        await bus.PublishAsync(new TenantCreatedIntegrationEvent
        {
            NewTenantId = tenant.Id,
            TenantId = tenant.Id,
            Name = tenant.Name,
            Subdomain = tenant.Subdomain,
            AdminEmail = cmd.AdminEmail
        });

        // 5) Devolver la respuesta.
        return Result.Success(
            new TenantResponse(tenant.Id, tenant.Name, tenant.Subdomain));
    }
}

```

La query y su handler

```
src/Services/Tenant/TaxVision.Tenant.Application/Tenants/Queries/GetTenants.cs
```

```
using Microsoft.EntityFrameworkCore;
using TaxVision.Tenant.Application.Tenants.Commands;
using TaxVision.Tenant.Infrastructure.Persistence;

namespace TaxVision.Tenant.Application.Tenants.Queries;

// QUERY: solo lee, no cambia estado. Paginada.
public sealed record GetTenantsQuery(int Page = 1, int Size = 20);

public static class GetTenantsHandler
{
    public static async Task<IReadOnlyList<TenantResponse>> Handle(
        GetTenantsQuery q,
        TenantDbContext db,           // lectura directa, sin repositorio
        CancellationToken ct)
    {
        return await db.Tenants
            .AsNoTracking()           // solo lectura: sin tracking, más rápido
            .OrderBy(t => t.Name)
            .Skip((q.Page - 1) * q.Size)
            .Take(q.Size)
            .Select(t => new TenantResponse(t.Id, t.Name, t.Subdomain))
            .ToListAsync(ct);
    }
}
```

Nota sobre la referencia a Infrastructure desde la query. Aquí la query de lectura usa `TenantDbContext` directamente (camino de lectura optimizado de CQRS). Para que Application "vea" el DbContext sin invertir la dependencia limpia, en TaxVision colocamos el `DbContext` en Infrastructure y la query lo recibe por inyección — Application define la query, pero el tipo concreto se resuelve en tiempo de ejecución desde Infrastructure, que sí referencia a Application. Si prefieres pureza estricta, define una interfaz de lectura en Application; aquí elegimos el pragmatismo del camino de lectura.

4.4 La capa Infrastructure (EF Core)

► Paso 1 — Instalar paquetes de EF Core

```
$ cd src/Services/Tenant/TaxVision.Tenant.Infrastructure
$ dotnet add package Microsoft.EntityFrameworkCore.SqlServer
$ dotnet add package Microsoft.EntityFrameworkCore.Design
$ mkdir -p Persistence/Configurations Persistence/Repositories
$ cd ../../../../..
```

La configuración de la entidad

src/Services/Tenant/TaxVision.Tenant.Infrastructure/Persistence/Configurations/
TenantConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using DomainTenant = TaxVision.Tenant.Domain.Tenants.Tenant;

namespace TaxVision.Tenant.Infrastructure.Persistence.Configurations;

// Configura cómo se mapea la entidad Tenant a la tabla SQL.
public sealed class TenantConfiguration : IEntityTypeConfiguration<DomainTenant>
{
    public void Configure(EntityTypeBuilder<DomainTenant> b)
    {
        b.ToTable("Tenants");
        b.HasKey(t => t.Id);

        b.Property(t => t.Name).HasMaxLength(200).IsRequired();
        b.Property(t => t.Subdomain).HasMaxLength(40).IsRequired();
        b.Property(t => t.Status).HasConversion<string>(); // enum como texto legible

        // El subdominio es único globalmente (a diferencia del email por tenant).
        b.HasIndex(t => t.Subdomain).IsUnique();

        // Ignorar la colección de eventos: no es una columna, es estado en memoria.
        b.Ignore(t => t.DomainEvents);
    }
}
```

El DbContext

src/Services/Tenant/TaxVision.Tenant.Infrastructure/Persistence/TenantDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using System.Reflection;
using DomainTenant = TaxVision.Tenant.Domain.Tenants.Tenant;

namespace TaxVision.Tenant.Infrastructure.Persistence;

// El DbContext es el Unit of Work de EF Core: agrupa cambios en una transacción.
public sealed class TenantDbContext(DbContextOptions<TenantDbContext> options)
    : DbContext(options)
{
    public DbSet<DomainTenant> Tenants => Set<DomainTenant>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Aplica TODAS las IEntityTypeConfiguration de este ensamblado.
        modelBuilder.ApplyConfigurationsFromAssembly(Assembly.GetExecutingAssembly());
        base.OnModelCreating(modelBuilder);
    }
}
```

¿Por qué el Tenant Service no tiene filtro global por TenantId? Porque **gestiona los tenants en sí**: su tabla `Tenants` contiene a todos. Es el único servicio así. Los demás (Customer, Billing...) sí aplicarán el filtro global por `TenantId` que veremos en la Sección 6, porque sus datos pertenecen a tenants. El Tenant Service es la "oficina de registro", no un inquilino.

El repositorio concreto

```
src/Services/Tenant/TaxVision.Tenant.Infrastructure/Persistence/Repositories/
TenantRepository.cs

using Microsoft.EntityFrameworkCore;
using TaxVision.Tenant.Application.Abstractions;
using DomainTenant = TaxVision.Tenant.Domain.Tenants.Tenant;

namespace TaxVision.Tenant.Infrastructure.Persistence.Repositories;

// Implementación concreta del repositorio sobre EF Core.
public sealed class TenantRepository(TenantDbContext db) : ITenantRepository
{
    public Task<DomainTenant?> GetByIdAsync(Guid id, CancellationToken ct = default)
        => db.Tenants.FirstOrDefaultAsync(t => t.Id == id, ct);

    public async Task AddAsync(DomainTenant entity, CancellationToken ct = default)
        => await db.Tenants.AddAsync(entity, ct);

    public void Remove(DomainTenant entity) => db.Tenants.Remove(entity);

    public Task<bool> SubdomainExistsAsync(string subdomain, CancellationToken ct = default)
        => db.Tenants.AnyAsync(t => t.Subdomain == subdomain.ToLower(), ct);
}
```

Registro de la infraestructura

```
src/Services/Tenant/TaxVision.Tenant.Infrastructure/DependencyInjection.cs
```

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using TaxVision.Tenant.Application.Abstractions;
using TaxVision.Tenant.Infrastructure.Persistence;
using TaxVision.Tenant.Infrastructure.Persistence.Repositories;

namespace TaxVision.Tenant.Infrastructure;

public static class InfrastructureRegistration
{
    public static IServiceCollection AddTenantInfrastructure(
        this IServiceCollection services, IConfiguration config)
    {
        // Registrar el DbContext con la cadena de conexión.
        services.AddDbContext<TenantDbContext>(opt =>
            opt.UseSqlServer(config.GetConnectionString("Default")));

        // El repositorio concreto detrás de la interfaz de Application.
        services.AddScoped<ITenantRepository, TenantRepository>();
        return services;
    }
}

```

4.5 La capa Api (endpoints y arranque)

► Paso 1 — Instalar paquetes de la API

```

$ cd src/Services/Tenant/TaxVision.Tenant.Api
$ dotnet add package WolverineFx
$ dotnet add package Serilog.AspNetCore
$ mkdir Endpoints
$ cd ../../../../..

```

Los endpoints (Minimal API)

```
src/Services/Tenant/TaxVision.Tenant.Api/Endpoints/TenantEndpoints.cs
```

```

using BuildingBlocks.Results;
using TaxVision.Tenant.Application.Tenants.Commands;
using TaxVision.Tenant.Application.Tenants.Queries;
using Wolverine;

namespace TaxVision.Tenant.Api.Endpoints;

public static class TenantEndpoints
{
    public static void MapTenantEndpoints(this IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/tenants");

        // POST /tenants → crear un tenant. El endpoint SOLO traduce HTTP a mensaje.
        group.MapPost("/", async (CreateTenantCommand cmd, IMessageBus bus) =>
        {
            var result = await bus.InvokeAsync<Result<TenantResponse>>(cmd);
            return result.IsSuccess
                ? Results.Created($"tenants/{result.Value.Id}", result.Value)
                : Results.BadRequest(result.Error);
        });

        // GET /tenants → listar (paginado).
        group.MapGet("/", async (IMessageBus bus, int page = 1, int size = 20) =>
            Results.Ok(await bus.InvokeAsync<IReadOnlyList<TenantResponse>>(
                new GetTenantsQuery(page, size)))));
    }
}

```

El Program.cs (composición final)

```
src/Services/Tenant/TaxVision.Tenant.Api/Program.cs
```

```

using BuildingBlocks.Common;
using BuildingBlocks.Middleware;
using Serilog;
using TaxVision.Tenant.Api.Endpoints;
using TaxVision.Tenant.Infrastructure;
using Wolverine;

var builder = WebApplication.CreateBuilder(args);

// 1) Logging estructurado con Serilog.
builder.Host.UseSerilog((ctx, cfg) => cfg
    .Enrich.FromLogContext()
    .Enrich.WithProperty("service", "tenant-service")
    .WriteTo.Console());

// 2) Servicios compartidos + infraestructura del servicio.
builder.Services.AddBuildingBlocks();
builder.Services.AddTenantInfrastructure(builder.Configuration);

// 3) Wolverine: mediador (CQRS) y bus de mensajería.
// En la sección 9 le añadiremos RabbitMQ + Outbox. Por ahora, en memoria.
builder.Host.UseWolverine();

var app = builder.Build();

// 4) Pipeline de middlewares (¡el ORDEN importa!).
app.UseMiddleware<CorrelationIdMiddleware>(); // primero: id para todo
app.UseMiddleware<ExceptionHandlerMiddleware>(); // envuelve el resto
app.UseMiddleware<TenantResolutionMiddleware>(); // resuelve el tenant

// 5) Health checks para Kubernetes (sección 12).
app.MapGet("/health/live", () => Results.Ok("alive"));

// 6) Endpoints del servicio.
app.MapTenantEndpoints();

app.Run();

```

La cadena de conexión

src/Services/Tenant/TaxVision.Tenant.Api/appsettings.json

```

{
  "ConnectionStrings": {
    // En desarrollo apunta a SQL Server en Docker (sección 11).
    "Default": "Server=localhost,1433;Database=TaxVision_Tenants;User
Id=sa;Password=Your_Strong_Pass123;TrustServerCertificate=true"
  },
  "Logging": { "LogLevel": { "Default": "Information" } }
}

```

► Verificar que todo compila

```
# Desde la raíz: compilar toda la solución
$ dotnet build
Build succeeded. 0 Warning(s) 0 Error(s)
```

¿Por qué? — ¿Por qué el endpoint usa `bus.InvokeAsync` y no llama al handler directamente? Porque desacopla la API de la implementación. El endpoint solo sabe "quiero ejecutar este command y obtener un Result"; Wolverine se encarga de encontrar el handler, inyectarle sus dependencias y ejecutarlo. Mañana puedes añadir comportamiento transversal (validación, logging, transacciones) en el pipeline de Wolverine sin tocar un solo endpoint. La API queda delgada: traduce HTTP a mensaje y mensaje a HTTP, nada más.

Resumen de la sección 4

- Creamos los **4 proyectos** del Tenant Service y configuramos las referencias **apuntando al dominio**.
- **Domain:** el aggregate `Tenant` con fábrica que valida, setters privados, comportamiento con intención de negocio y eventos de dominio.
- **Application:** el `CreateTenantCommand` + handler (con Wolverine y publicación del evento de integración) y la `GetTenantsQuery` optimizada para lectura.
- **Infrastructure:** `TenantDbContext`, la configuración EF de la entidad y el repositorio concreto.
- **Api:** endpoints Minimal API delgados, el `Program.cs` que compone todo y el pipeline de middlewares en orden.
- Este es **el patrón que replicarán todos los demás servicios**.

SECCIÓN 5

Auth Service (identidad y JWT)

El Auth Service es la fuente única de identidad de TaxVision: registra usuarios, los autentica, emite los JWT en los que confía todo el sistema, gestiona refresh tokens y administra roles y permisos. Aquí nace el `tenant_id` firmado que viaja en cada petición.

Reutilizamos el patrón de la Sección 4 (los mismos 4 proyectos), así que iré más rápido en lo estructural y me detendré en lo nuevo: el hash de contraseñas, la emisión del JWT y los refresh tokens.

5.1 Crear los proyectos

► Paso 1 — Crear los 4 proyectos y referencias (igual que Tenant)

```
# Desde la raíz TaxVision/
$ dotnet new classlib -n TaxVision.Auth.Domain -o src/Services/Auth/
TaxVision.Auth.Domain
$ dotnet new classlib -n TaxVision.Auth.Application -o src/Services/Auth/
TaxVision.Auth.Application
$ dotnet new classlib -n TaxVision.Auth.Infrastructure -o src/Services/Auth/
TaxVision.Auth.Infrastructure
$ dotnet new web -n TaxVision.Auth.Api -o src/Services/Auth/TaxVision.Auth.Api

# Borrar clases de ejemplo y agregar a la solución
$ rm src/Services/Auth/TaxVision.Auth.{Domain,Application,Infrastructure}/Class1.cs
$ dotnet sln add src/Services/Auth/**/*.csproj

# Referencias (apuntan al dominio)
$ dotnet add src/Services/Auth/TaxVision.Auth.Domain reference src/BuildingBlocks/
BuildingBlocks.csproj
$ dotnet add src/Services/Auth/TaxVision.Auth.Application reference src/Services/Auth/
TaxVision.Auth.Domain
$ dotnet add src/Services/Auth/TaxVision.Auth.Infrastructure reference src/Services/Auth/
TaxVision.Auth.Application
$ dotnet add src/Services/Auth/TaxVision.Auth.Api reference src/Services/Auth/
TaxVision.Auth.Application
$ dotnet add src/Services/Auth/TaxVision.Auth.Api reference src/Services/Auth/
TaxVision.Auth.Infrastructure
```

Atajo con glob. `dotnet sln add src/Services/Auth/**/*.csproj` agrega los cuatro proyectos de golpe. Si tu shell no expande `**`, agrégalos uno a uno como en la Sección 4.

5.2 Domain: el aggregate User

El `User` pertenece a un tenant, guarda el *hash* de la contraseña (nunca la contraseña), y tiene una lista de roles.

```
$ cd src/Services/Auth/TaxVision.Auth.Domain
$ mkdir -p Users/Events
$ cd ../../../../..
```

src/Services/Auth/TaxVision.Auth.Domain/Users/User.cs

```
using BuildingBlocks.Domain;
using BuildingBlocks.Results;
using TaxVision.Auth.Domain.Users.Events;

namespace TaxVision.Auth.Domain.Users;

public sealed class User : BaseEntity, ITenantOwned
{
    private readonly List<string> _roles = new();

    public Guid TenantId { get; private set; }
    public string Email { get; private set; } = default!;
    public string PasswordHash { get; private set; } = default!;
    public bool IsActive { get; private set; }
    public IReadOnlyCollection<string> Roles => _roles.AsReadOnly();

    private User() { }

    // El handler ya recibe el HASH (calculado en Infrastructure), no la clave.
    public static Result<User> Register(Guid tenantId, string email, string passwordHash)
    {
        if (string.IsNullOrEmpty(email) || !email.Contains('@'))
            return Result.Failure<User>(new Error("User.Email", "Email inválido."));

        var user = new User
        {
            TenantId = tenantId,
            Email = email.Trim().ToLowerInvariant(),
            PasswordHash = passwordHash,
            IsActive = true
        };
        user._roles.Add("User"); // rol base por defecto
        user.Raise(new UserRegisteredDomainEvent(user.Id, tenantId));
        return Result.Success(user);
    }

    public void AssignRole(string role)
    {
        if (!_roles.Contains(role)) _roles.Add(role);
    }

    void ITenantOwned.SetTenant(Guid id) => TenantId = id;
}
```

src/Services/Auth/TaxVision.Auth.Domain/Users/Events/UserRegisteredDomainEvent.cs

```
using BuildingBlocks.Domain;

namespace TaxVision.Auth.Domain.Users.Events;

public sealed record UserRegisteredDomainEvent(Guid UserId, Guid TenantId)
    : IDomainEvent
{
    public DateTime OccurredOn { get; } = DateTime.UtcNow;
}
```

5.3 Infrastructure: hash de contraseñas y JWT

► Paso 1 — Instalar paquetes

```
$ cd src/Services/Auth/TaxVision.Auth.Infrastructure
$ dotnet add package Microsoft.EntityFrameworkCore.SqlServer
$ dotnet add package Microsoft.EntityFrameworkCore.Design
$ dotnet add package Microsoft.AspNetCore.Cryptography.KeyDerivation
$ dotnet add package System.IdentityModel.Tokens.Jwt
$ mkdir -p Security/Persistence/Configurations Persistence/Repositories
$ cd ../../../../..
```

El hasher de contraseñas

Primero el contrato en Application, luego la implementación en Infrastructure (que usa PBKDF2 con sal por usuario).

src/Services/Auth/TaxVision.Auth.Application/Abstractions/IPasswordHasher.cs

```
namespace TaxVision.Auth.Application.Abstractions;

public interface IPasswordHasher
{
    string Hash(string password);
    bool Verify(string password, string hash);
}
```

src/Services/Auth/TaxVision.Auth.Infrastructure/Security/Pbkdf2PasswordHasher.cs

```

using System.Security.Cryptography;
using Microsoft.AspNetCore.Cryptography.KeyDerivation;
using TaxVision.Auth.Application.Abstractions;

namespace TaxVision.Auth.Infrastructure.Security;

// Deriva la contraseña con PBKDF2 + sal aleatoria por usuario.
// Formato almacenado: {salt-base64}.{hash-base64}
public sealed class Pbkdf2PasswordHasher : IPasswordHasher
{
    private const int SaltSize = 16, KeySize = 32, Iterations = 100_000;

    public string Hash(string password)
    {
        var salt = RandomNumberGenerator.GetBytes(SaltSize);
        var hash = KeyDerivation.Pbkdf2(
            password, salt, KeyDerivationPrf.HMACSHA256, Iterations, KeySize);
        return $"{Convert.ToBase64String(salt)}.{Convert.ToBase64String(hash)}";
    }

    public bool Verify(string password, string stored)
    {
        var parts = stored.Split('.');
        if (parts.Length != 2) return false;

        var salt = Convert.FromBase64String(parts[0]);
        var expected = Convert.FromBase64String(parts[1]);
        var actual = KeyDerivation.Pbkdf2(
            password, salt, KeyDerivationPrf.HMACSHA256, Iterations, KeySize);

        // Comparación en tiempo constante: evita ataques de temporización.
        return CryptographicOperations.FixedTimeEquals(actual, expected);
    }
}

```

¿Por qué? — ¿Por qué PBKDF2 con 100.000 iteraciones y comparación en tiempo constante? Las iteraciones hacen el hash deliberadamente lento, lo que frustra los ataques de fuerza bruta (probar millones de contraseñas). La sal aleatoria por usuario impide usar tablas precalculadas (rainbow tables). Y `FixedTimeEquals` compara siempre en el mismo tiempo, sin "cortocircuitar" al primer byte distinto, para que un atacante no pueda deducir el hash midiendo cuánto tarda la comparación. Nunca uses `==` para comparar hashes de seguridad.

El generador de JWT

```
src/Services/Auth/TaxVision.Auth.Infrastructure.Security/JwtTokenGenerator.cs
```

```

using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using TaxVision.Auth.Application.Abstractions;
using TaxVision.Auth.Domain.Users;

namespace TaxVision.Auth.Infrastructure.Security;

// Opciones leídas de appsettings (sección "Jwt").
public sealed class JwtOptions
{
    public string Secret { get; set; } = default!;
    public string Issuer { get; set; } = default!;
    public string Audience { get; set; } = default!;
    public int AccessMinutes { get; set; } = 15;
}

public sealed class JwtTokenGenerator(IOptions<JwtOptions> options) : IJwtTokenGenerator
{
    private readonly JwtOptions _opt = options.Value;

    public string Generate(User user, IEnumerable<string> permissions)
    {
        var claims = new List<Claim>
        {
            new(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
            new(JwtRegisteredClaimNames.Email, user.Email),
            // CLAVE: el tenant viaja FIRMADO dentro del token.
            new("tenant_id", user.TenantId.ToString())
        };
        claims.AddRange(user.Roles.Select(r => new Claim(ClaimTypes.Role, r)));
        claims.AddRange(permissions.Select(p => new Claim("permission", p)));

        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_opt.Secret));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _opt.Issuer, audience: _opt.Audience, claims: claims,
            expires: DateTime.UtcNow.AddMinutes(_opt.AccessMinutes),
            signingCredentials: creds);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}

```

El contrato correspondiente, en Application:

```
src/Services/Auth/TaxVision.Auth.Application/Abstractions/IJwtTokenGenerator.cs
```

```
using TaxVision.Auth.Domain.Users;

namespace TaxVision.Auth.Application.Abstractions;

public interface IJwtTokenGenerator
{
    string Generate(User user, IEnumerable<string> permissions);
}
```

5.4 Application: registro y login

Dos casos de uso clave. El registro crea el usuario; el login verifica credenciales y emite el JWT más un refresh token.

`src/Services/Auth/TaxVision.Auth.Application/Users/Commands/Login.cs`

```
using BuildingBlocks.Results;
using Microsoft.EntityFrameworkCore;
using TaxVision.Auth.Application.Abstractions;
using TaxVision.Auth.Infrastructure.Persistence;

namespace TaxVision.Auth.Application.Users.Commands;

public sealed record LoginCommand(string Email, string Password);
public sealed record LoginResponse(string AccessToken, string RefreshToken);

public static class LoginHandler
{
    public static async Task<Result<LoginResponse>> Handle(
        LoginCommand cmd,
        AuthDbContext db,
        IPasswordHasher hasher,
        IJwtTokenGenerator jwt,
        IRefreshTokenService refresh,
        CancellationToken ct)
    {
        var email = cmd.Email.Trim().ToLowerInvariant();
        var user = await db.Users.FirstOrDefaultAsync(u => u.Email == email, ct);

        // Mensaje genérico: no revelar si el fallo fue el email o la clave.
        if (user is null || !hasher.Verify(cmd.Password, user.PasswordHash))
            return Result.Failure<LoginResponse>(
                new Error("Auth.Invalid", "Credenciales inválidas."));

        if (!user.IsActive)
            return Result.Failure<LoginResponse>(
                new Error("Auth.Inactive", "Usuario inactivo."));

        var access = jwt.Generate(user, permissions: user.Roles); // simplificado
        var refreshToken = await refresh.IssueAsync(user.Id, ct);

        return Result.Success(new LoginResponse(access, refreshToken));
    }
}
```

El mensaje de error que protege la privacidad. Fíjate en que, tanto si el email no existe como si la contraseña es incorrecta, devolvemos el mismo error genérico "Credenciales inválidas". Si dijéramos "ese email no existe", un atacante podría *enumerar* qué emails están registrados. La ambigüedad es deliberada y es una buena práctica de seguridad.

5.5 Refresh tokens

El refresh token se guarda **hasheado** y es revocable. El servicio que lo emite y valida vive en Infrastructure; aquí la idea clave (el código completo sigue el mismo patrón de hash que las contraseñas): se genera un token aleatorio, se entrega al cliente en claro, y en la base de datos solo queda su hash con fecha de expiración y un flag de revocado. Al renovar, se busca por hash, se valida que no esté expirado ni revocado, se revoca el usado y se emite uno nuevo (rotación de refresh tokens).

¿Por qué? — ¿Por qué dos tokens en vez de uno largo? El access token es de vida corta (15 min): si se filtra, el daño caduca pronto, y como se valida solo por su firma, ningún servicio necesita consultar a Auth en cada petición (escala bien). El refresh token es de vida larga pero *revocable* y se guarda hasheado: permite renovar sin re-login, y ante un robo o un logout, se revoca al instante. Juntos equilibran seguridad y experiencia de usuario.

Resumen de la sección 5

- Auth Service replica los **4 proyectos**; el aggregate `User` guarda el hash, nunca la contraseña.
- El **hash PBKDF2** con sal e iteraciones, y comparación en **tiempo constante**, frustran fuerza bruta y ataques de temporización.
- El **JWT** lleva el `tenant_id` firmado y los permisos; se valida por firma sin consultar a Auth.
- El **login** devuelve un error genérico para no filtrar qué emails existen.
- El **access token** es corto; el **refresh token**, largo, hasheado, revocable y rotatorio.

SECCIÓN 6

Customer Service (filtro multitenant)

*El Customer Service es el núcleo del CRM y, a diferencia del Tenant Service, sus datos **pertenecen a tenants**. Por eso aquí construimos la pieza central del aislamiento: el filtro global de EF Core que garantiza que un tenant jamás vea datos de otro. Esta sección es la que debes interiorizar a fondo.*

6.1 Crear los proyectos y el dominio

► Paso 1 — Los 4 proyectos (igual patrón)

```
$ dotnet new classlib -n TaxVision.Customer.Domain -o src/Services/Customer/
TaxVision.Customer.Domain
$ dotnet new classlib -n TaxVision.Customer.Application -o src/Services/Customer/
TaxVision.Customer.Application
$ dotnet new classlib -n TaxVision.Customer.Infrastructure -o src/Services/Customer/
TaxVision.Customer.Infrastructure
$ dotnet new web -n TaxVision.Customer.Api -o src/Services/Customer/
TaxVision.Customer.Api
$ rm src/Services/Customer/TaxVision.Customer.{Domain,Application,Infrastructure}/Class1.cs
$ dotnet sln add src/Services/Customer/**/*.csproj
# ...y las mismas referencias apuntando al dominio (ver Sección 4.1)
```

El value object Email y el aggregate Customer

src/Services/Customer/TaxVision.Customer.Domain/Customers/Email.cs

```
using BuildingBlocks.Results;

namespace TaxVision.Customer.Domain.Customers;

// VALUE OBJECT: sin identidad, inmutable, autovalidado. Un Email inválido
// es imposible de construir → "estados inválidos irrepresentables".
public sealed record Email
{
    public string Value { get; }
    private Email(string value) => Value = value;

    public static Result<Email> Create(string? input)
    {
        if (string.IsNullOrEmpty(input) || !input.Contains('@'))
            return Result.Failure<Email>(new Error("Email.Invalid", "Email inválido."));
        return Result.Success(new Email(input.Trim().ToLowerInvariant()));
    }
}
```


src/Services/Customer/TaxVision.Customer.Domain/Customers/Customer.cs

```

using BuildingBlocks.Domain;
using BuildingBlocks.Results;
using TaxVision.Customer.Domain.Customers.Events;

namespace TaxVision.Customer.Domain.Customers;

public sealed class Customer : BaseEntity, ITenantOwned
{
    public Guid TenantId { get; private set; }
    public string FullName { get; private set; } = default!;
    public Email Email { get; private set; } = default!;
    public bool IsActive { get; private set; }

    private Customer() { }

    public static Result<Customer> Create(string fullName, Email email)
    {
        if (string.IsNullOrWhiteSpace(fullName))
            return Result.Failure<Customer>(new Error("Customer.Name", "Nombre obligatorio."));

        var c = new Customer { FullName = fullName.Trim(), Email = email, IsActive = true };
        c.Raise(new CustomerCreatedDomainEvent(c.Id));
        return Result.Success(c);
    }

    // EF asigna el TenantId vía esta interfaz (lo hace el DbContext al guardar).
    void ITenantOwned.SetTenant(Guid id) => TenantId = id;
}

```

6.2 El corazón del aislamiento: filtro global + estampado

Aquí está la pieza más importante de toda la guía en términos de seguridad. El `DbContext` del Customer Service hace dos cosas automáticas que ningún desarrollador debe recordar manualmente: **filtra toda lectura por el tenant actual** y **estampa el tenant en toda escritura**.

src/Services/Customer/TaxVision.Customer.Infrastructure/Persistence/CustomerDbContext.cs

```

using System.Reflection;
using BuildingBlocks.Domain;
using BuildingBlocks.Tenancy;
using Microsoft.EntityFrameworkCore;
using TaxVision.Customer.Domain.Customers;

namespace TaxVision.Customer.Infrastructure.Persistence;

public sealed class CustomerDbContext : DbContext
{
    private readonly ITenantContext _tenant;

    // El ITenantContext se inyecta: trae el tenant de la petición actual.
    public CustomerDbContext(
        DbContextOptions<CustomerDbContext> options, ITenantContext tenant)
        : base(options) => _tenant = tenant;

    public DbSet<Customer> Customers => Set<Customer>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        b.ApplyConfigurationsFromAssembly(Assembly.GetExecutingAssembly());

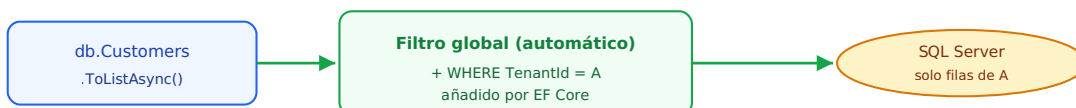
        // + FILTRO GLOBAL: toda consulta a Customers trae SOLO el tenant actual.
        // Se aplica automáticamente a TODA query, sin que nadie tenga que recordarlo.
        b.Entity<Customer>()
            .HasQueryFilter(c => c.TenantId == _tenant.TenantId);

        base.OnModelCreating(b);
    }

    // + ESTAMPADO: al INSERTAR una entidad de tenant, le pone el TenantId actual.
    public override Task<int> SaveChangesAsync(CancellationToken ct = default)
    {
        foreach (var entry in ChangeTracker.Entries<ITenantOwned>())
        {
            if (entry.State == EntityState.Added)
                entry.Entity.SetTenant(_tenant.TenantId);
        }
        return base.SaveChangesAsync(ct);
    }
}

```

Petición del Tenant A consulta "todos los clientes"



Resultado: A jamás recibe filas de B, aunque la query no mencione el tenant.

El desarrollador escribe ``db.Customers.ToListAsync()`` sin pensar en el tenant;
EF Core inyecta el ``WHERE TenantId = ...`` en CADA consulta de forma transparente.

Figura 6.1 — El filtro global convierte cada consulta en una consulta segura por tenant, automáticamente.

Las dos reglas que protegen el aislamiento — y cómo se rompen. (1) **Nunca** uses `IgnoreQueryFilters()` en código de aplicación: desactiva el filtro y expone datos de todos los tenants. (2) **Nunca** aceptes el `TenantId` desde el cuerpo de la petición: siempre viene del `ITenantContext` (que el middleware llenó desde el token). Si en una revisión de código ves cualquiera de las dos cosas, es un fallo de seguridad, no un detalle de estilo.

Configuración EF con índice e índice único por tenant

`src/Services/Customer/TaxVision.Customer.Infrastructure/Persistence/Configurations/`
`CustomerConfiguration.cs`

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using TaxVision.Customer.Domain.Customers;

namespace TaxVision.Customer.Infrastructure.Persistence.Configurations;

public sealed class CustomerConfiguration : IEntityTypeConfiguration<Customer>
{
    public void Configure(EntityTypeBuilder<Customer> b)
    {
        b.ToTable("Customers");
        b.HasKey(c => c.Id);
        b.Property(c => c.FullName).HasMaxLength(200).IsRequired();

        // Value object Email → columna plana mediante conversión.
        b.Property(c => c.Email)
            .HasConversion(e => e.Value, v => Email.Create(v).Value)
            .HasColumnName("Email").HasMaxLength(256);

        // Índice con TenantId PRIMERO: casi toda query filtra por tenant.
        b.HasIndex(c => c.TenantId);

        // Unicidad de email POR TENANT (no global): dos tenants distintos
        // pueden tener clientes con el mismo email.
        b.HasIndex(c => new { c.TenantId, c.Email }).IsUnique();

        b.Ignore(c => c.DomainEvents);
    }
}
```

¿Por qué? — ¿Por qué el índice lleva `TenantId` primero y la unicidad es por tenant? Como cada consulta filtra por tenant, poner `TenantId` como primera columna del índice hace que SQL Server localice las filas del tenant de inmediato (sin escanear toda la tabla). Y la unicidad del email debe ser *dentro* de cada tenant: si fuera global, dos oficinas distintas no podrían tener, ambas, un cliente "juan@gmail.com" — lo cual sería absurdo, porque son clientes de negocios diferentes que casualmente comparten dirección.

6.3 El registro de infraestructura (con tenant)

A diferencia del Tenant Service, aquí el `DbContext` depende de `ITenantContext`, que ya viene registrado por `AddBuildingBlocks()`. El registro es casi idéntico:

`src/Services/Customer/TaxVision.Customer.Infrastructure/DependencyInjection.cs`

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using TaxVision.Customer.Application.Abstractions;
using TaxVision.Customer.Infrastructure.Persistence;
using TaxVision.Customer.Infrastructure.Persistence.Repositories;

namespace TaxVision.Customer.Infrastructure;

public static class InfrastructureRegistration
{
    public static IServiceCollection AddCustomerInfrastructure(
        this IServiceCollection services, IConfiguration config)
    {
        // El DbContext recibirá ITenantContext por inyección automáticamente.
        services.AddDbContext<CustomerDbContext>(opt =>
            opt.UseSqlServer(config.GetConnectionString("Default")));

        services.AddScoped<ICustomerRepository, CustomerRepository>();
        return services;
    }
}
```

Recuerda registrar el pipeline en Program.cs. En el `Program.cs` del Customer Service, el middleware `TenantResolutionMiddleware` debe ejecutarse **antes** de cualquier endpoint, para que el `ITenantContext` ya tenga el tenant cuando el `DbContext` se use. El orden es el mismo de la Sección 4.5: Correlation → Exception → Tenant → endpoints.

Resumen de la sección 6

- Customer Service replica el patrón, pero sus datos **pertenecen a tenants** (implementa `ITenantOwned`).
- El `Email` es un **value object** autovalidado; un email inválido es imposible de construir.
- El `CustomerDbContext` aplica dos automatismos críticos: **filtro global por tenant** en lectura y **estampado del tenant** en escritura.
- Prohibido `IgnoreQueryFilters()` y aceptar el `TenantId` del cuerpo: ambos rompen el aislamiento.
- Índices con `TenantId` **primero** y unicidad de email **por tenant**.

SECCIÓN 7

Subscription, Notification y Billing

Los tres servicios restantes siguen exactamente el patrón de las secciones anteriores: cuatro proyectos, dominio con aggregates, CQRS en Application e EF Core en Infrastructure. Aquí me centro en lo que cada uno aporta de nuevo, no en repetir la mecánica que ya domina.

7.1 Crear los tres servicios de golpe

Como la estructura es idéntica, los creamos en una sola tanda. Repite el bloque cambiando el nombre del servicio:

```
# Plantilla: sustituye {Svc} por Subscription, Notification y Billing
$ dotnet new classlib -n TaxVision.{Svc}.Domain -o src/Services/{Svc}/TaxVision.
{Svc}.Domain
$ dotnet new classlib -n TaxVision.{Svc}.Application -o src/Services/{Svc}/TaxVision.
{Svc}.Application
$ dotnet new classlib -n TaxVision.{Svc}.Infrastructure -o src/Services/{Svc}/TaxVision.
{Svc}.Infrastructure
$ dotnet new web -n TaxVision.{Svc}.Api -o src/Services/{Svc}/TaxVision.
{Svc}.Api
$ dotnet sln add src/Services/{Svc}/**/*.csproj
# ...y las mismas referencias apuntando al dominio (Sección 4.1)
```

Automatízalo con un script. Crea un pequeño script de shell que reciba el nombre del servicio y ejecute estos comandos. Construir un microservicio nuevo se reduce así a una línea, garantizando que todos nacen idénticos en estructura. Es la ventaja de tener un patrón consistente.

7.2 Subscription Service: planes y proration

Lo nuevo aquí es el aggregate `Subscription` con cambio de plan y la lógica de prorrateo. Es un `ITenantOwned` (sus datos pertenecen a tenants), así que usa el mismo `DbContext` con filtro global de la Sección 6.

```
src/Services/Subscription/TaxVision.Subscription.Domain/Subscriptions/Subscription.cs
```

```

using BuildingBlocks.Domain;
using BuildingBlocks.Results;

namespace TaxVision.Subscription.Domain.Subscriptions;

public enum SubscriptionStatus { Trial, Active, PastDue, Cancelled }

public sealed class Subscription : BaseEntity, ITenantOwned
{
    public Guid TenantId { get; private set; }
    public Guid PlanId { get; private set; }
    public SubscriptionStatus Status { get; private set; }
    public DateOnly CurrentPeriodEnd { get; private set; }

    private Subscription() { }

    // Al crear el tenant, se le asigna un plan trial automáticamente.
    public static Subscription StartTrial(Guid tenantId, Guid planId, int trialDays)
        => new()
        {
            TenantId = tenantId, PlanId = planId,
            Status = SubscriptionStatus.Trial,
            CurrentPeriodEnd = DateOnly.FromDateTime(DateTime.UtcNow.AddDays(trialDays))
        };

    public Result ChangePlan(Guid newPlanId)
    {
        if (Status == SubscriptionStatus.Cancelled)
            return Result.Failure(new Error("Sub.Cancelled", "Suscripción cancelada."));
        PlanId = newPlanId;
        return Result.Success();
    }

    void ITenantOwned.SetTenant(Guid id) => TenantId = id;
}

```

7.3 Notification Service: consumidor puro

Notification casi no tiene endpoints de escritura: su superficie son **consumidores de eventos**. Lo nuevo es la abstracción de canales y un consumidor real. Veremos cómo Wolverine enruta los eventos a estos handlers en la Sección 9.

`src/Services/Notification/TaxVision.Notification.Application/Abstractions/EmailSender.cs`

```

namespace TaxVision.Notification.Application.Abstractions;

// Contrato del canal. La lógica de negocio no conoce el proveedor concreto.
public interface IEmailSender
{
    Task SendAsync(string to, string subject, string body, CancellationToken ct);
}

```

`src/Services/Notification/TaxVision.Notification.Application/Consumers/TenantCreatedConsumer.cs`

```
using TaxVision.Notification.Application.Abstractions;
// El tipo del evento lo comparte el servicio emisor (contrato público).
using TaxVision.Tenant.Application.Tenants.IntegrationEvents;

namespace TaxVision.Notification.Application.Consumers;

// CONSUMIDOR: Wolverine descubre este handler y lo suscribe a la cola
// del evento TenantCreated. Notification reacciona, no decide.
public static class TenantCreatedConsumer
{
    public static async Task Handle(
        TenantCreatedIntegrationEvent evt,
        IEmailSender email,
        CancellationToken ct)
    {
        // Idempotente: si el evento llega duplicado, enviar el mismo email
        // no causa un daño grave; el Inbox de Wolverine ya filtra reentregas.
        await email.SendAsync(
            to: evt.AdminEmail,
            subject: "Bienvenido a TaxVision",
            body: $"Tu oficina '{evt.Name}' ya está activa en {evt.Subdomain}.taxvision.com",
            ct);
    }
}
```

¿Por qué? — ¿Por qué Notification no llama a nadie? Es un servicio "tonto" a propósito. No decide *cuándo* notificar; reacciona a eventos que otros publican. Esto concentra toda la complejidad de envío (plantillas, proveedores, reintentos) en un solo lugar y desacopla a los demás servicios: cuando Tenant crea un tenant, no sabe ni le importa que se envíe un email. Si mañana añades notificaciones por SMS, solo tocas Notification.

7.4 Billing Service: idempotencia con dinero

Lo crítico de Billing es la idempotencia estricta: maneja dinero, y un evento procesado dos veces no debe cobrar dos veces. Usa un value object `Money` e `Invoice` como agregate.

```
src/Services/Billing/TaxVision.Billing.Application/Consumers/PaymentReceivedConsumer.cs
```

```
namespace TaxVision.Billing.Application.Consumers;

public static class PaymentReceivedConsumer
{
    public static async Task Handle(
        PaymentReceivedIntegrationEvent evt,
        IInvoiceRepository repo,
        IUnitOfWork uow,
        CancellationToken ct)
    {
        var invoice = await repo.GetByIdAsync(evt.InvoiceId, ct);
        if (invoice is null) return;

        // + IDEMPOTENCIA DE NEGOCIO: si ya está pagada, NO hacer nada.
        // Sin esto, un reenvío del broker duplicaría el asiento contable.
        if (invoice.Status == InvoiceStatus.Paid) return;

        invoice.MarkAsPaid(evt.PaidAt);
        await uow.SaveChangesAsync(ct);
    }
}
```

El error que cuesta dinero. RabbitMQ entrega "al menos una vez": el mismo evento puede llegar duplicado. Sin la comprobación `if (Status == Paid) return;`, un reintento generaría un segundo cobro. En Billing, **toda operación con efecto monetario verifica el estado antes de actuar**. La idempotencia no es opcional aquí.

Resumen de la sección 7

- Subscription, Notification y Billing repiten el **patrón de 4 proyectos**; automatízalo con un script.
- **Subscription** aporta planes, trial y cambio de plan; es `ITenantOwned`.
- **Notification** es un **consumidor puro**: reacciona a eventos, con canales tras interfaces.
- **Billing** exige **idempotencia estricta**: verifica el estado antes de toda operación monetaria.

SECCIÓN 8

API Gateway con YARP

El gateway es la única puerta pública de TaxVision. Centraliza la validación del JWT, el rate limiting, CORS y —la pieza clave— la propagación del `tenant_id` hacia los microservicios. Lo construimos con YARP, configurable casi por completo desde `appsettings.json`.

8.1 Crear el proyecto del gateway

► Paso 1 — Crear el proyecto e instalar YARP

```
$ dotnet new web -n TaxVision.Gateway -o src/Gateway/TaxVision.Gateway
$ dotnet sln add src/Gateway/TaxVision.Gateway/TaxVision.Gateway.csproj
$ cd src/Gateway/TaxVision.Gateway

# El reverse proxy de Microsoft
$ dotnet add package Yarp.ReverseProxy
$ dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
$ dotnet add package BuildingBlocks # si lo expones como paquete, o referencia el csproj
$ dotnet add reference ../../BuildingBlocks/BuildingBlocks.csproj
$ mkdir Middleware
$ cd ../../..
```

8.2 Configurar rutas y clusters

YARP enruta con dos conceptos: **routes** (qué petición va a dónde) y **clusters** (los destinos, con balanceo y health checks). Toda la configuración va en `appsettings.json`.

```
src/Gateway/TaxVision.Gateway/appsettings.json
```

```

{
  "Jwt": {
    "Issuer": "https://auth.taxvision.com",
    "Audience": "taxvision",
    "Secret": "clave-de-32-bytes-o-mas-igual-que-en-auth"
  },
  "ReverseProxy": {
    "Routes": {
      // Auth es público (login no requiere estar autenticado).
      "auth-route": {
        "ClusterId": "auth-cluster",
        "Match": { "Path": "/auth/{**catch-all}" }
      },
      // El resto exige autenticación.
      "tenant-route": {
        "ClusterId": "tenant-cluster",
        "AuthorizationPolicy": "authenticated",
        "Match": { "Path": "/tenants/{**catch-all}" }
      },
      "customer-route": {
        "ClusterId": "customer-cluster",
        "AuthorizationPolicy": "authenticated",
        "Match": { "Path": "/customers/{**catch-all}" }
      }
    },
    "Clusters": {
      "auth-cluster": {
        "Destinations": {
          "d1": { "Address": "http://auth-service:8080/" }
        }
      },
      "tenant-cluster": {
        "Destinations": {
          "d1": { "Address": "http://tenant-service:8080/" }
        }
      },
      "customer-cluster": {
        "LoadBalancingPolicy": "RoundRobin",
        "HealthCheck": {
          "Active": { "Enabled": true, "Path": "/health/live",
            "Interval": "00:00:10" }
        },
        "Destinations": {
          // dos réplicas: YARP balancea entre ellas
          "d1": { "Address": "http://customer-service-1:8080/" },
          "d2": { "Address": "http://customer-service-2:8080/" }
        }
      }
    }
  }
}

```

¿Por qué? — ¿Por qué los destinos usan nombres como `customer-service` y no IPs? En Docker y Kubernetes, los servicios se resuelven por nombre en la red interna. `http://customer-service:8080` funciona porque el orquestador tiene un DNS interno que traduce ese nombre a la(s) instancia(s) reales. Así el gateway nunca necesita saber IPs concretas, y cuando Kubernetes escala el servicio a más réplicas, el balanceo sigue funcionando sin tocar la configuración.

8.3 La propagación del tenant (la pieza clave de seguridad)

El gateway valida el JWT, extrae el claim `tenant_id` y lo inyecta como cabecera `X-Tenant-Id` hacia el servicio destino — **borrando antes** cualquier cabecera que el cliente hubiera intentado colar.

`src/Gateway/TaxVision.Gateway/Middleware/TenantPropagationMiddleware.cs`

```
using Microsoft.AspNetCore.Http;

namespace TaxVision.Gateway.Middleware;

public sealed class TenantPropagationMiddleware(RequestDelegate next)
{
    public async Task InvokeAsync(HttpContext ctx)
    {
        // ¡ SIEMPRE borrar la cabecera entrante: el cliente NO puede fijarla.
        ctx.Request.Headers.Remove("X-Tenant-Id");

        // Tomar el tenant del TOKEN firmado (fuente de confianza) y reescribirlo.
        var tenantId = ctx.User.FindFirst("tenant_id")?.Value;
        if (!string.IsNullOrEmpty(tenantId))
            ctx.Request.Headers["X-Tenant-Id"] = tenantId;

        await next(ctx);
    }
}
```

El agujero de seguridad que esto cierra. Si confiáramos en una `X-Tenant-Id` enviada por el cliente, un atacante la cambiaría y accedería a otro tenant. La defensa es doble: el gateway **borra y reescribe** la cabecera desde el token firmado, y los microservicios **no son accesibles desde fuera** (viven en la red interna de Docker/K8s, solo el gateway los alcanza). Borrar primero es tan importante como reescribir: si solo reescribieras cuando hay token, una petición sin token podría colar una cabecera falsa.

8.4 El Program.cs del gateway

src/Gateway/TaxVision.Gateway/Program.cs

```
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using TaxVision.Gateway.Middleware;

var builder = WebApplication.CreateBuilder(args);

// 1) YARP lee Routes/Clusters de la configuración.
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));

// 2) Validación de JWT (misma clave/issuer que Auth Service).
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new()
        {
            ValidateIssuer = true, ValidateAudience = true,
            ValidateLifetime = true, ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Secret"]!))
        };
    });
builder.Services.AddAuthorization(o =>
    o.AddPolicy("authenticated", p => p.RequireAuthenticatedUser()));

// 3) CORS: solo el frontend Angular conocido.
builder.Services.AddCors(o => o.AddDefaultPolicy(p => p
    .WithOrigins("https://app.taxvision.com")
    .AllowAnyHeader().AllowAnyMethod()));

// 4) Rate limiting por tenant.
builder.Services.AddRateLimiter(o =>
    o.AddFixedWindowLimiter("per-tenant", opt =>
    {
        opt.PermitLimit = 100;
        opt.Window = TimeSpan.FromMinutes(1);
    }));

var app = builder.Build();

// † El ORDEN del pipeline: CORS → auth → rate limit → tenant → proxy.
app.UseCors();
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();
app.UseMiddleware<TenantPropagationMiddleware>(); // tras autenticar
app.MapReverseProxy();

app.Run();
```

El orden no es negociable. `TenantPropagationMiddleware` debe ir **después** de `UseAuthentication`, porque necesita que el JWT ya esté validado y los claims disponibles en `ctx.User`. Si lo pusieras antes, no habría `tenant_id` que leer. Cada middleware depende de lo que los anteriores dejaron preparado.

Resumen de la sección 8

- El gateway YARP es la **única puerta pública**; se configura con **routes y clusters** en `appsettings.json`.
- Valida **JWT**, aplica **CORS y rate limiting**, y enruta a los servicios por nombre de red interna.
- El `TenantPropagationMiddleware` **borra y reescribe** `X-Tenant-Id` desde el token: cierra el agujero de suplantación de tenant.
- El **orden del pipeline** (CORS → auth → rate limit → tenant → proxy) es obligatorio.

SECCIÓN 9

Wolverine + RabbitMQ (eventos entre servicios)

Hasta ahora Wolverine actuó solo como mediador en memoria (despachando commands a handlers). Ahora le añadimos RabbitMQ para que los servicios se comuniquen por eventos, con Outbox e Inbox que garantizan que ningún evento se pierda ni se procese dos veces.

9.1 Paquetes necesarios

En cada servicio que publique o consuma eventos (todos menos el gateway), añade los paquetes de Wolverine para RabbitMQ y la persistencia con SQL Server.

```
# En cada *.Api que use mensajería (ej. Tenant)
$ cd src/Services/Tenant/TaxVision.Tenant.Api
$ dotnet add package WolverineFx.RabbitMQ
$ dotnet add package WolverineFx.SqlServer
$ cd ../../../../..
```

9.2 Configurar Wolverine con RabbitMQ y Outbox

Esta configuración reemplaza el `UseWolverine()` simple que pusimos en la Sección 4. Va en el `Program.cs` del servicio.

```
src/Services/Tenant/TaxVision.Tenant.Api/Program.cs (bloque de Wolverine)
```

```

using Wolverine;
using Wolverine.RabbitMQ;
using Wolverine.SqlServer;
using TaxVision.Tenant.Application.Tenants.IntegrationEvents;

builder.Host.UseWolverine(opts =>
{
    var rabbitUri = builder.Configuration["RabbitMq:Uri"]!;
    var sqlConn    = builder.Configuration.GetConnectionString("Default")!;

    // 1) Conexión a RabbitMQ; crea exchanges/colas si no existen.
    opts.UseRabbitMq(new Uri(rabbitUri)).AutoProvision();

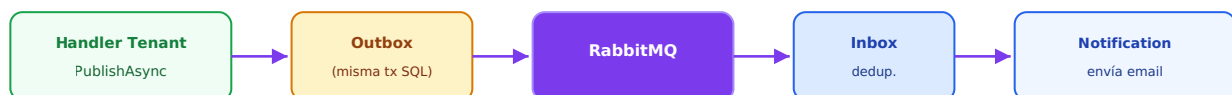
    // 2) Persistencia para Outbox/Inbox sobre la MISMA base del servicio.
    opts.PersistMessagesWithSqlServer(sqlConn);

    // 3) Outbox automático en todos los envíos: el evento se guarda en la
    //     misma transacción que el dato, y se publica DESPUÉS del commit.
    opts.Policies.UseDurableOutboxOnAllSendingEndpoints();

    // 4) Publicar TenantCreated hacia un exchange compartido.
    opts.PublishMessage<TenantCreatedIntegrationEvent>()
        .ToRabbitExchange("taxvision-events");

    // 5) Reintentos con backoff ante fallos transitorios del broker.
    opts.Policies.OnException<Exception>()
        .RetryWithCooldown(
            TimeSpan.FromSeconds(1),
            TimeSpan.FromSeconds(5),
            TimeSpan.FromSeconds(15));
});

```



Wolverine gestiona Outbox e Inbox de forma transparente: tú solo publicas y consumes.

Figura 9.1 — Camino completo de un evento con Wolverine: Outbox transaccional → RabbitMQ → Inbox idempotente → handler.

9.3 Consumir el evento en otro servicio

En Notification (y cualquier consumidor), la configuración de Wolverine es la misma, pero en lugar de `PublishMessage` se *escucha* la cola. Wolverine descubre el handler `TenantCreatedConsumer.Handle` (Sección 7.3) por convención y lo suscribe automáticamente. Solo hay que indicarle de qué cola/exchange leer:

```
src/Services/Notification/TaxVision.Notification.Api/Program.cs (bloque de escucha)
```

```
builder.Host.UseWolverine(opts =>
{
    opts.UseRabbitMq(new Uri(rabbitUri)).AutoProvision();
    opts.PersistMessagesWithSqlServer(sqlConn);

    // Escuchar el exchange compartido: Wolverine enruta cada evento
    // al handler cuyo parámetro coincide con el tipo del mensaje.
    opts.ListenToRabbitQueue("notification-queue")
        .UseDurableInbox(); // idempotencia: descarta reentregas
});
```

9.4 Por qué Outbox e Inbox importan tanto

¿Por qué? — ¿Por qué no publicar directamente al broker en el handler?

Porque crearías el problema de la "doble escritura": si guardas el tenant en SQL y luego publicas en RabbitMQ, ¿qué pasa si la base confirma pero el broker está caído? El tenant existe pero nadie se entera. El **Outbox** lo resuelve: el evento se guarda en una tabla *dentro de la misma transacción* que el dato, y un proceso de Wolverine lo publica después. Si el broker está caído, el evento espera en la tabla; nunca se pierde. El **Inbox** hace lo simétrico en el consumidor: registra qué eventos ya procesó para descartar duplicados, porque RabbitMQ entrega "al menos una vez". Wolverine implementa ambos por ti; solo tienes que habilitarlos.

Resumen de la sección 9

- Wolverine pasa de mediador en memoria a **bus de eventos** al añadir `WolverineFx.RabbitMQ` y `WolverineFx.SqlServer`.
- `UseDurableOutboxOnAllSendingEndpoints` activa el **Outbox**: el evento se guarda con el dato en la misma transacción y se publica tras el commit.
- Los consumidores usan `ListenToRabbitQueue(...).UseDurableInbox()` para **idempotencia** automática.
- Publicar es `PublishAsync`; consumir es un handler por convención. Wolverine oculta toda la complejidad de Outbox/Inbox.

SECCIÓN 10

Migraciones de base de datos

Cada microservicio gestiona su propia base de datos y sus propias migraciones, de forma independiente. Nunca hay una "migración global". En esta sección creamos y aplicamos las migraciones con la CLI de EF Core, y vemos cómo hacerlo bien en desarrollo y en producción.

10.1 Crear la primera migración de un servicio

La herramienta `dotnet ef` necesita saber dos cosas: dónde está el `DbContext` (el proyecto de Infrastructure) y desde dónde arrancar la app para resolver la configuración (el proyecto de Api).

► Paso 1 — Asegúrate de tener SQL Server levantado

Las migraciones se aplican contra una base real. Levantaremos SQL Server con Docker en la Sección 11; por ahora basta con que la cadena de conexión de `appsettings.json` apunte a una instancia accesible.

► Paso 2 — Crear la migración inicial (ejemplo: Tenant)

```
# Desde la raíz TaxVision/
$ dotnet ef migrations add InitialCreate \
  --project src/Services/Tenant/TaxVision.Tenant.Infrastructure \
  --startup-project src/Services/Tenant/TaxVision.Tenant.Api \
  --output-dir Persistence/Migrations

Build started... Done. To undo this action, use 'ef migrations remove'
```

Esto genera, dentro de Infrastructure, las clases de migración:

```
TaxVision.Tenant.Infrastructure/
├── Persistence/
│   └── Migrations/
│       ├── 20250115_InitialCreate.cs
│       ├── 20250115_InitialCreate.Designer.cs
│       └── TenantDbContextModelSnapshot.cs
```

Up() y Down(): crea/revierte la tabla

estado actual del modelo

► Paso 3 — Aplicar la migración a la base de datos

```
$ dotnet ef database update \
'
  --project src/Services/Tenant/TaxVision.Tenant.Infrastructure \
  --startup-project src/Services/Tenant/TaxVision.Tenant.Api

Applying migration '20250115_InitialCreate'. Done.
```

¿Por qué? — ¿Por qué `--project` y `--startup-project` separados? El `DbContext` y las migraciones viven en **Infrastructure** (ahí está el modelo de datos). Pero la configuración —la cadena de conexión que está en `appsettings.json`— vive en **Api**. EF Core necesita "arrancar" la app desde **Api** para leer esa configuración y saber a qué base conectarse, mientras genera el código de migración en **Infrastructure**. Por eso se indican los dos.

10.2 El flujo de trabajo con migraciones

Cada vez que cambias el modelo (añades una propiedad, una entidad, un índice), el ciclo es siempre el mismo:



Figura 10.1 — Ciclo de migración: cambiar el modelo, generar, revisar el script generado y aplicar.

Revisa siempre el script generado antes de aplicarlo. EF Core es bueno, pero no infalible. El paso 3 —abrir el `.cs` de la migración y leer qué `Up()` va a ejecutar— evita sorpresas: renombrados que EF interpreta como "borrar columna + crear otra" (perdiendo datos), cambios de tipo arriesgados, etc. En producción, este script es lo que tocará tu base real.

10.3 Migraciones en producción

En desarrollo aplicas con `database update`. En producción **nunca** migres automáticamente al arrancar el servicio: si hay varias réplicas, competirían por migrar a la vez. En su lugar, genera un script SQL idempotente o usa un Job de Kubernetes (Sección 12) que migre *una vez* antes de levantar las réplicas nuevas.

```
# Generar un script SQL idempotente para revisión/despliegue manual
$ dotnet ef migrations script --idempotent \
  --project src/Services/Tenant/TaxVision.Tenant.Infrastructure \
  --startup-project src/Services/Tenant/TaxVision.Tenant.Api \
  --output deploy/migrations/tenant.sql
```

Repíte por cada servicio. Tenant, Auth, Customer, Subscription y Billing tienen cada uno su propio `DbContext` y base de datos, así que cada uno tiene su propio juego de migraciones. Notification, al ser un consumidor casi sin estado propio, puede no tener tablas de negocio (solo las del Inbox de Wolverine, que este crea por su cuenta).

Resumen de la sección 10

- Cada servicio tiene sus **propias migraciones** en su Infrastructure; no hay migración global.
- Se crean con `dotnet ef migrations add`, indicando `--project` (Infrastructure) y `--startup-project` (Api).
- El ciclo es: cambiar el modelo → generar → **revisar el script** → aplicar.
- En producción, **no migres al arrancar**: usa un script idempotente o un Job, antes de levantar las réplicas.

SECCIÓN 11

Docker: empaquetar y levantar todo

Ahora empaquetamos cada servicio en una imagen Docker y levantamos todo el entorno —SQL Server, RabbitMQ, los microservicios y el gateway— con un solo comando. Esto convierte "clonar el repo y arrancar el sistema completo" en algo de treinta segundos.

11.1 El Dockerfile de un microservicio

Usamos una construcción *multi-stage*: una etapa con el SDK compila, y la imagen final solo lleva el runtime y lo publicado. Crea este archivo en la carpeta de cada servicio (ejemplo con Tenant).

src/Services/Tenant/Dockerfile

```
# ---- Etapa 1: build con el SDK de .NET 10 ----
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

# Copiar los .csproj primero y restaurar: aprovecha la caché de capas.
COPY ["src/BuildingBlocks/BuildingBlocks.csproj", "src/BuildingBlocks/"]
COPY ["src/Services/Tenant/TaxVision.Tenant.Api/TaxVision.Tenant.Api.csproj", "src/Services/
Tenant/TaxVision.Tenant.Api/"]
COPY ["src/Services/Tenant/TaxVision.Tenant.Application/TaxVision.Tenant.Application.csproj",
"src/Services/Tenant/TaxVision.Tenant.Application/"]
COPY ["src/Services/Tenant/TaxVision.Tenant.Infrastructure/
TaxVision.Tenant.Infrastructure.csproj", "src/Services/Tenant/TaxVision.Tenant.Infrastructure/"]
COPY ["src/Services/Tenant/TaxVision.Tenant.Domain/TaxVision.Tenant.Domain.csproj", "src/
Services/Tenant/TaxVision.Tenant.Domain/"]
RUN dotnet restore "src/Services/Tenant/TaxVision.Tenant.Api/TaxVision.Tenant.Api.csproj"

# Ahora el resto del código y publicar en Release.
COPY . .
RUN dotnet publish "src/Services/Tenant/TaxVision.Tenant.Api/TaxVision.Tenant.Api.csproj" \
    -c Release -o /app/publish /p:UseAppHost=false

# ---- Etapa 2: runtime mínimo ----
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app

# Ejecutar como usuario NO root (buena práctica de seguridad).
RUN adduser --disabled-password --gecos "" appuser
USER appuser

COPY --from=build /app/publish .
EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080
ENTRYPOINT ["dotnet", "TaxVision.Tenant.Api.dll"]
```

¿Por qué? — ¿Por qué copiar los .csproj y restaurar antes que el resto del código? Docker construye en capas y cachea cada una. Si copias solo los `.csproj` y haces `restore` primero, mientras no cambien las dependencias Docker reutiliza esa capa (la más lenta) y solo recompila tu código cuando lo cambias. Sin este truco, cada build descargaría todos los paquetes de cero. Y ejecutar como `appuser` (no `root`) limita el daño si el contenedor se ve comprometido.

11.2 El docker-compose para desarrollo

Este archivo levanta el sistema completo. Va en `deploy/docker/`. Incluye la infraestructura (SQL Server, RabbitMQ, Redis) y los servicios, todos en una red interna donde solo el gateway expone puerto público.

```
deploy/docker/docker-compose.yml
```

```

services:
# ----- Infraestructura -----
sqlserver:
  image: mcr.microsoft.com/mssql/server:2022-latest
  environment:
    SA_PASSWORD: "${SA_PASSWORD}"      # viene del archivo .env
    ACCEPT_EULA: "Y"
  ports: ["1433:1433"]
  volumes: ["sqldata:/var/opt/mssql"]
  networks: ["backend"]

rabbitmq:
  image: rabbitmq:3-management
  ports: ["5672:5672", "15672:15672"]  # broker + panel web
  networks: ["backend"]

redis:
  image: redis:7-alpine
  networks: ["backend"]

# ----- Microservicios -----
tenant-service:
  build:
    context: ../../                  # raíz del repo (para ver src/)
    dockerfile: src/Services/Tenant/Dockerfile
  environment:
    ConnectionStrings__Default: "Server=sqlserver;Database=TaxVision_Tenants;User
Id=sa;Password=${SA_PASSWORD};TrustServerCertificate=true"
    RabbitMq__Uri: "amqp://guest:guest@rabbitmq:5672"
    depends_on: ["sqlserver", "rabbitmq"]
    networks: ["backend"]          # SIN ports: no es público

auth-service:
  build:
    context: ../../
    dockerfile: src/Services/Auth/Dockerfile
  environment:
    ConnectionStrings__Default: "Server=sqlserver;Database=TaxVision_Auth;User
Id=sa;Password=${SA_PASSWORD};TrustServerCertificate=true"
    RabbitMq__Uri: "amqp://guest:guest@rabbitmq:5672"
    Jwt__Secret: "${JWT_SECRET}"
    Jwt__Issuer: "https://auth.taxvision.com"
    Jwt__Audience: "taxvision"
    depends_on: ["sqlserver", "rabbitmq"]
    networks: ["backend"]

customer-service:
  build:
    context: ../../
    dockerfile: src/Services/Customer/Dockerfile
  environment:
    ConnectionStrings__Default: "Server=sqlserver;Database=TaxVision_Customers;User
Id=sa;Password=${SA_PASSWORD};TrustServerCertificate=true"
    RabbitMq__Uri: "amqp://guest:guest@rabbitmq:5672"
    depends_on: ["sqlserver", "rabbitmq"]
    networks: ["backend"]

# ----- Gateway: la ÚNICA puerta pública -----
gateway:
  build:

```

```

    context: ../../
    dockerfile: src/Gateway/TaxVision.Gateway/Dockerfile
    environment:
      Jwt__Secret: "${JWT_SECRET}"
      Jwt__Issuer: "https://auth.taxvision.com"
      Jwt__Audience: "taxvision"
    ports: ["8080:8080"]           # el único con puerto público
    depends_on: ["tenant-service", "auth-service", "customer-service"]
    networks: ["backend"]

networks:
  backend: {}

volumes:
  sqldata: {}

```

El archivo .env (no se versiona)

TaxVision/.env

```

# Secretos locales. Este archivo está en .gitignore.
SA_PASSWORD=Your_Strong_Pass123
JWT_SECRET=una-clave-larga-y-aleatoria-de-al-menos-32-bytes

```

11.3 Levantar y verificar

► Levantar todo el sistema

```

# Desde deploy/docker/ – construye y levanta todo
$ cd deploy/docker
$ docker compose up --build

# O en segundo plano (detached)
$ docker compose up --build -d

# Ver el estado de los contenedores
$ docker compose ps

# Ver logs de un servicio concreto
$ docker compose logs -f gateway

# Detener y limpiar todo
$ docker compose down

```

Verificación rápida. Con el sistema arriba, el panel de RabbitMQ está en <http://localhost:15672> (usuario/clave: guest/guest) y la API responde a través del gateway en <http://localhost:8080>. Prueba a crear un tenant con `POST http://localhost:8080/tenants` usando el REST Client de VS Code, y observa en los logs cómo Notification consume el evento `TenantCreated`.

Resumen de la sección 11

- Cada servicio tiene un **Dockerfile multi-stage**: compila con el SDK, entrega una imagen mínima como usuario no root.
- **docker-compose** levanta todo el entorno (SQL Server, RabbitMQ, Redis, servicios y gateway) en una red interna.
- Solo el **gateway expone puerto público**; los servicios se ven entre sí por nombre.
- Los secretos van en `.env` (ignorado por git), inyectados como variables de entorno.

SECCIÓN 12

Kubernetes: desplegar en producción

Docker empaqueta y prueba; Kubernetes despliega en producción: escala, reinicia y conecta los servicios. En esta sección final escribimos los manifiestos para desplegar TaxVision en un clúster, con configuración, secretos, health checks, escalado automático y migraciones como Job.

12.1 Namespace, ConfigMap y Secret

Todo va en un namespace propio. La configuración no sensible en un ConfigMap; los secretos, separados en un Secret.

deploy/k8s/00-namespace-config.yaml

```
apiVersion: v1
kind: Namespace
metadata: { name: taxvision }
---
apiVersion: v1
kind: ConfigMap
metadata: { name: shared-config, namespace: taxvision }
data:
  RabbitMq__Uri: "amqp://rabbitmq:5672"
  Jwt__Issuer: "https://auth.taxvision.com"
  Jwt__Audience: "taxvision"
---
apiVersion: v1
kind: Secret
metadata: { name: shared-secrets, namespace: taxvision }
type: Opaque
stringData:
  Jwt__Secret: "clave-larga-y-aleatoria-de-al-menos-32-bytes"
  ConnectionStrings__Customers: "Server=sql.taxvision.com;Database=TaxVision_Customers;..."
```

Los Secrets de Kubernetes no están cifrados por defecto. Solo están codificados en base64, no cifrados. Para producción real, habilita el cifrado en reposo del clúster o usa un gestor de secretos externo. Y **nunca** subas este YAML con valores reales al repositorio: versiona una plantilla con valores de ejemplo.

12.2 Deployment y Service de un microservicio

Cada servicio se despliega con un Deployment (réplicas, imagen, probes, recursos) y un Service (dirección estable interna). Ejemplo con Customer y 3 réplicas:

deploy/k8s/customer.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata: { name: customer-service, namespace: taxvision }
spec:
  replicas: 3
  selector:
    matchLabels: { app: customer-service }
  template:
    metadata:
      labels: { app: customer-service }
    spec:
      containers:
        - name: customer-service
          image: registry.taxvision.com/customer-service:1.0.0 # versión fija
          ports: [{ containerPort: 8080 }]
          envFrom:
            - configMapRef: { name: shared-config }
            - secretRef: { name: shared-secrets }
          # Readiness: ¿listo para recibir tráfico? (verifica dependencias)
          readinessProbe:
            httpGet: { path: /health/ready, port: 8080 }
            initialDelaySeconds: 5
            periodSeconds: 10
          # Liveness: ¿sigue vivo? (si falla, K8s reinicia el Pod)
          livenessProbe:
            httpGet: { path: /health/live, port: 8080 }
            initialDelaySeconds: 10
            periodSeconds: 15
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "512Mi" }
      ---
apiVersion: v1
kind: Service
metadata: { name: customer-service, namespace: taxvision }
spec:
  selector: { app: customer-service }
  ports: [{ port: 8080, targetPort: 8080 }]
  # ClusterIP por defecto: accesible solo DENTRO del clúster.

```

Recuerda añadir el endpoint `/health/ready` en el `Program.cs` del servicio (en la Sección 4.5 solo pusimos `/health/live`):

...Customer.Api/Program.cs (health ready)

```

// Readiness verifica que la base de datos responde.
builder.Services.AddHealthChecks()
    .AddSqlServer(builder.Configuration.GetConnectionString("Default")!,
        name: "sql", tags: ["ready"]);

app.MapHealthChecks("/health/live", new() { Predicate = _ => false });
app.MapHealthChecks("/health/ready",
    new() { Predicate = c => c.Tags.Contains("ready") });

```

¿Por qué? — ¿Por qué dos probes distintas? **Liveness** responde "¿el proceso está colgado?": si falla, K8s reinicia el Pod. **Readiness** responde "¿puede atender tráfico ahora?": si falla (p. ej. la base está momentáneamente lenta), K8s lo saca del balanceo pero *no* lo reinicia. La distinción evita reiniciar un Pod sano solo porque su base de datos tuvo un hipo: lo sacas del tráfico hasta que se recupere, sin matarlo.

12.3 El gateway y el Ingress

El gateway es el único expuesto al exterior, a través de un Ingress que termina TLS y enruta el tráfico público hacia él.

deploy/k8s/gateway-ingress.yaml

```
# (El Deployment y Service del gateway son como los de customer.yaml)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: taxvision-ingress
  namespace: taxvision
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod" # TLS automático
spec:
  tls:
    - hosts: ["api.taxvision.com"]
      secretName: taxvision-tls
  rules:
    - host: api.taxvision.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: gateway # todo el tráfico va al gateway
                port: { number: 8080 }
```

12.4 Escalado automático (HPA)

El HorizontalPodAutoscaler ajusta las réplicas según la carga. Recuerda: esto funciona porque los servicios son **sin estado** (el estado vive en la base de datos o Redis, nunca en memoria del Pod).

deploy/k8s/customer-hpa.yaml

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: customer-hpa, namespace: taxvision }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: customer-service
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }

```

12.5 Migraciones como Job

Como dijimos en la Sección 10, en producción la migración se aplica con un Job que corre una vez, antes de levantar las réplicas nuevas.

deploy/k8s/customer-migrate-job.yaml

```

apiVersion: batch/v1
kind: Job
metadata: { name: customer-migrate-1-0-0, namespace: taxvision }
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: registry.taxvision.com/customer-service:1.0.0
          args: ["--migrate"] # el servicio detecta este flag y migra
          envFrom:
            - secretRef: { name: shared-secrets }

```

12.6 Aplicar todo al clúster

```

# Aplicar todos los manifiestos de la carpeta
$ kubectl apply -f deploy/k8s/

# Ver el estado de los pods
$ kubectl get pods -n taxvision

# Ver logs de un servicio
$ kubectl logs -n taxvision -l app=customer-service -f

# Escalar manualmente (el HPA lo hará solo bajo carga)
$ kubectl scale deployment customer-service -n taxvision --replicas=5

```

Resumen de la sección 12

- Todo vive en un **namespace**; la config en **ConfigMap**, los secretos en **Secret** (cifrar en producción).
- Cada servicio: **Deployment + Service** con probes de **liveness y readiness** y límites de recursos.
- El **gateway** es el único expuesto, vía **Ingress** con TLS.
- El **HPA** escala según CPU, gracias a que los servicios son **sin estado**.
- Las **migraciones** se aplican con un **Job** antes de levantar las réplicas.

CIERRE

Lo que has construido

Si has seguido la guía de principio a fin, ahora tienes TaxVision funcionando: un CRM multitenant de microservicios en .NET 10, construido con tus propias manos, desplegable de tu máquina al clúster.

Recapitulemos el camino. Preparaste el entorno en VS Code y creaste la solución y su estructura. Construiste **BuildingBlocks**, el cimiento compartido. Levantaste el **Tenant Service** de extremo a extremo —que se convirtió en la plantilla de todos los demás— y luego **Auth** (con JWT, hash de contraseñas y refresh tokens), **Customer** (con el filtro multitenant que aísla a cada tenant), y **Subscription, Notification y Billing**. Uniste todo tras un **API Gateway con YARP** y conectaste los servicios con eventos sobre **Wolverine y RabbitMQ**, con Outbox e Inbox. Creaste las **migraciones**, empaquetaste con **Docker** y desplegaste con **Kubernetes**.

El orden recomendado para construirlo desde cero

Si vuelves a empezar o incorporas a alguien al equipo, este es el orden que minimiza el "no compila todavía":

1. **BuildingBlocks** primero: todo depende de él.
2. **Tenant Service** completo: aprende el patrón con un servicio sin la complejidad del filtro multitenant.
3. **Auth Service**: necesario para que el resto tenga tokens con qué autenticarse.
4. **Customer Service**: aquí interiorizas el filtro global multitenant.
5. **Gateway**: ya puedes probar el flujo real (login → token → petición con tenant).
6. **Wolverine + RabbitMQ**: conecta los servicios por eventos.
7. **Resto de servicios**: Subscription, Notification, Billing.
8. **Docker**, luego **Kubernetes**: empaqueta y despliega.

Una última recomendación. No intentes construir los seis servicios a la vez. Haz que **uno** funcione completo —de la petición HTTP al evento consumido por otro— antes de replicar el patrón. Cuando ese primer flujo de extremo a extremo funcione, los demás servicios son repeticiones con variaciones, y avanzarás mucho más rápido y con menos frustración. Construye en vertical (un flujo completo), no en horizontal (todas las capas de todos los servicios a la vez).

— *Fin de la guía* —

TaxVision · Guía de Implementación · Stack Vision Software
Santiago de los Caballeros, República Dominicana